

-
-
-
-
-
-
-
-

tema 3. Especificación de nivel lógico-RT de sistemas HW usando VHDL

Diseño Automático de Sistemas

José Manuel Mendías Cuadros
Dpto. Arquitectura de Computadores y Automática
Universidad Complutense de Madrid

-
-
-
-
-
-
-

2

VHDL “sintetizable”

- ☒ VHDL es un lenguaje de modelado, no de especificación
 - *Todo lo que existe puede ser modelado, pero no todo lo que se especifica puede ser sintetizado*
 - Muchas de las construcciones VHDL, aunque sean útiles para simulación y otras etapas del ciclo de diseño, no son relevantes desde el punto de vista de la síntesis, por ello o bien no son soportadas, o bien sólo lo son parcialmente.
 - Desde el punto de vista de una herramienta de síntesis, las construcciones VHDL pueden clasificarse en:
 - ✓ **Soportadas:** especifican claramente una funcionalidad HW que debe sintetizarse.
 - ✓ **Ignoradas:** pueden encontrarse en el código fuente VHDL, pero no se tienen en cuenta durante el proceso de síntesis.
 - ✓ **Prohibidas:** no pueden aparecer en el código fuente VHDL, si lo hacen se aborta el proceso de síntesis.
 - El subconjunto sintetizable (construcciones soportadas) y su semántica puede variar de herramienta a herramienta
 - ✓ La semántica de simulación es estándar, la de síntesis no
- ☒ VHDL es un lenguaje orientado a hardware que permite escribir software
 - Es necesario cambiar de mentalidad, pensar en software solo trae problemas
- ☒ VHDL (y herramientas EDA) no evitan diseñar hasta un cierto nivel de abstracción
 - Antes de codificar, debe haberse diseñado el sistema
 - En muchos sentidos VHDL permite “capturar esquemas” textualmente

diseño automático de sistemas

-
-
-
-
-
-
-

unidades de diseño

☒ Entity

- El cuerpo de sentencias se ignora, tanto el declarativo como el ejecutivo.
- Los genéricos se soportan si son de tipo *integer*.
 - ✓ usar pares de funciones de codificación a *integer* y de decodificación al tipo deseado.
- Los valores iniciales de los puertos se ignoran.

☒ Architecture

- Se permiten múltiples arquitecturas por entidad pero sólo se sintetiza una.
- Se prohíbe la interacción mediante señales globales (declaradas en package).

☒ Configuration

- Se ignoran si especifican el nivel más alto de la jerarquía, en otro caso están prohibidas.
- Las configuraciones explícitas están prohibidas.
- El enlace se realiza mediante comandos específicos de la herramienta de síntesis.

☒ Package y package body

- Soportados con restricciones según sus contenidos.

☒ Bibliotecas

- La existencia de bibliotecas y la compilación separada de unidades está soportada.

diseño automático de sistemas

• • • • • • • •

sentencias concurrentes

☒ Process

- La lista de sensibilidad se ignora, pero para asegurar coherencia entre simulación y síntesis debe seguirse las normas habituales de modelado en VHDL.

☒ Block

- Los bloques guardados y los bloques con puertos están prohibidos.

☒ Otras sentencias concurrentes

- La llamada concurrente a procedimiento está soportada.
- La aserción concurrente es ignorada.
- Todos los tipos de asignación concurrente están soportados.
 - ✓ El modelo de retardo especificado en la asignación se ignora.
 - ✓ La asignación de formas de onda está prohibida.
- La instanciación de componentes está soportada.
- La generación está soportada.

diseño automático de sistemas

• • • • • • • •

sentencias secuenciales

- ☒ **Sentencias condicionales**
 - Las sentencias **if** y **case** están soportadas.
- ☒ **Bucles**
 - El bucle tipo **for** está soportado si el rango del índice es estático y el cuerpo no contiene sentencias *wait*.
 - Los bucles de tipo **while** o sin esquema de repetición están soportado si el cuerpo contiene alguna sentencia *wait*.
- ☒ **Asignación**
 - La asignación a variables está soportada
 - La asignación a señales está soportada.
 - ✓ El modelo de retardo especificado en una asignación de señal se ignora.
 - ✓ La asignación de formas de onda está prohibida.
- ☒ **Sentencia wait**
 - Sólo está soportada bajo las siguientes formas (todas equivalentes)
 - ✓ **wait until** *clock* = VALUE;
 - ✓ **wait until clock'event and** *clock* = VALUE;
 - ✓ **wait until not clock'stable and** *clock* = VALUE;
 - No pueden usarse en bucles *for* ni en subprogramas
- ☒ **Otras sentencias secuenciales**
 - Las aserciones secuenciales se ignoran.
 - La llamada secuencial a procedimiento está soportada.
 - Las sentencias **next**, **exit**, **return** y **null** están soportadas.

diseño automático de sistemas

• • • • • • • •

declaraciones (i): objetos

- ☒ **Constantes.**
 - Completamente soportadas.
 - Los genéricos se soportan si son de tipo entero.
- ☒ **Variables.**
 - El tipo debe ser globalmente estático.
 - Los valores iniciales se ignoran.
- ☒ **Señales.**
 - El tipo debe ser globalmente estático.
 - Los valores iniciales se ignoran.
 - Prohibidas de clases **bus** y **register**, y por consiguiente las especificaciones de desconexión.
 - Se permiten las señales resueltas siempre y cuando la función de resolución especifique lógica cableada o tri-estado.
 - ✓ El cuerpo de las funciones de resolución es ignorado, por ello debe estar marcado con directivas (i.e. *resolution_method*)
 - Los puertos con modos **buffer** y **linkage** se interpretan como **out** e **inout**, respectivamente.
- ☒ **Ficheros.**
 - Prohibidos.
- ☒ **Componentes**
 - Deben tener siempre el mismo nombre que alguna entidad válida (la asociación dentro de la herramienta se hace por nombre y no por configuración).

diseño automático de sistemas

• • • • • • • •

declaraciones (ii): tipos y otros

- ☒ La declaración completa de tipos y subtipos está soportada con restricciones.
- ☒ **Reales**
 - Las declaraciones de tipos reales se ignoran.
 - Su uso está prohibido, excepto como literales en atributos definidos por la herramienta.
- ☒ **Enteros**
 - No se soportan con precisión infinita, deben especificarse con un rango explícito.
 - Todo objeto de un tipo entero se representa mediante un vector de bits lo más pequeño posible tal que pueda representar todos los valores posibles del rango especificado.
 - Si el rango especificado incluye números negativos, la representación usada será complemento a 2, en otro caso binaria pura.
 - Los tipos predefinidos **natural** y **positive** están soportados.
- ☒ **Enumerados**
 - Completamente soportados.
 - Todo objeto de un tipo enumerado se representa mediante un vector de bits lo más pequeño posible tal que pueda representar todos los valores posibles del tipo.
 - Por defecto, cada uno de los literales del tipo se representa mediante la codificación en binario puro de su posición dentro de la declaración. Esta codificación puede alterarse utilizando directivas o atributos (i.e. *enum_encoding*).
 - Los tipos predefinidos **character**, **boolean** y **bit** están soportados.
- ☒ **Físicos**
 - Las declaraciones de tipos físicos se ignoran.
 - Su uso en especificaciones de retardo se ignoran, cualquier otro uso está prohibido.

diseño automático de sistemas

• • • • • • • •

declaraciones (iii): tipos y otros

- ☒ **Vectores**
 - Sólo se soportan vectores monodimensionales de cualquier tipo soportado, por lo que aunque estén prohibidos los multidimensionales, pueden utilizarse los vectores de vectores.
 - Sólo se soportan índices enteros.
 - Se soporta la declaración y uso de vectores limitados y no limitados.
 - Todos los atributos sobre vectores están soportados.
 - Recordar que los tipos predefinidos **string** y **bit_vector** son vectores.
- ☒ **Registros**
 - Completamente soportados.
- ☒ **Punteros y ficheros**
 - Las declaraciones de tipo fichero o puntero se ignoran.
 - Su uso está prohibido.
- ☒ **Alias**
 - Se ignoran.
- ☒ **Atributos**
 - La declaración está soportada y la especificación también siempre que no contenga **all** u **others**.
 - El uso de atributos declarados por el usuario está prohibido.
 - Sólo está soportado el uso de algunos atributos predefinidos por la herramienta

diseño automático de sistemas

• • • • • • • •

subprogramas

- ☒ Sólo pueden declararse en paquetes o en la zona declarativa de las arquitecturas.
- ☒ La sobrecarga de subprogramas (y de operadores) está soportada.
- ☒ No pueden contener sentencias *wait*, por lo que pueden considerarse como instancias de circuitos combinacionales.
- ☒ Se prohíben los valores por defecto de los parámetros
- ☒ La recursividad está prohibida si no está acotada por un valor estático.
- ☒ Las funciones de resolución se permiten únicamente cuando especifican lógica cableada o tri-estado

diseño automático de sistemas

• • • • • • • •

expresiones: operadores y literales

- ☒ **Operadores**
 - Están soportados los lógicos, relacionales, aditivos, signo aritmético, valor absoluto y concatenación.
 - Multiplicación, división y resto se soportan según licencias. Si tales licencias no están disponibles, el operando derecho debe ser una constante múltiplo de 2.
 - Exponenciación solamente se soporta cuando ambos operandos son constantes y el derecho es múltiplo de 2.
- ☒ **Literales**
 - Los literales numéricos (con o sin base) están soportados.
 - Los vectores, rangos o rebanadas nulos están prohibidos.
 - Los literales físicos se ignoran.
 - Los literales tipo cadena están soportados
 - Los literales agregados sólo se permiten sobre vectores

diseño automático de sistemas

• • • • • • • •

expresiones: nombres y otros

☒ Nombres

- Los identificadores simples están soportados.
- La selección de identificadores está prohibida a excepción de que aparezca en la sentencia **use**. Está prohibido puentear el ámbito de visibilidad de los identificadores.
- Los indexados y rebanados de identificadores están soportados con excepción de los que se hacen sobre parámetros de salida de tipo vectorial no restringido. Deben ser estáticos.
- Los identificadores atribuidos están soportados con algunas restricciones:
 - ✓ Los atributos predefinidos **base**, **left**, **right**, **high**, **low**, **range**, **reverse_range**, y **length**, están completamente soportados.
 - ✓ Los atributos predefinidos **event** y **stable** están soportados con restricciones.
 - ✓ Los atributos definidos por el usuario están prohibidos.
 - ✓ Los atributos definidos por la herramienta están completamente soportados.

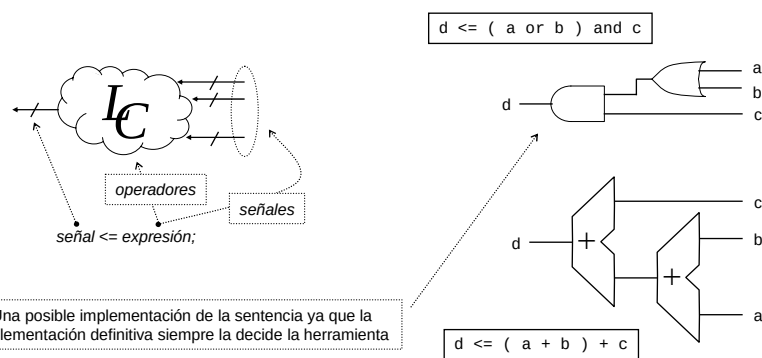
☒ Calificación y conversión

- Completamente soportadas

implementación de asignaciones concurrentes (i)

☒ Toda **asignación de señal** (o su proceso equivalente) se implementa como un bloque de lógica combinacional:

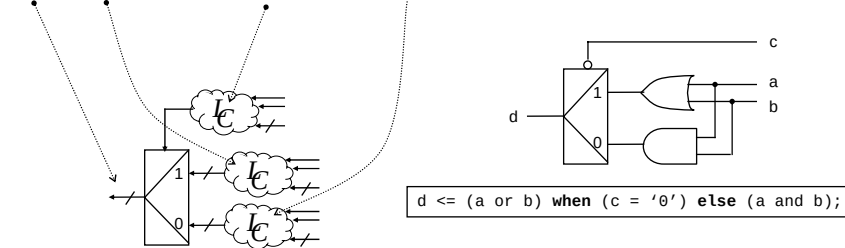
- Con un único puerto de salida (que puede ser vectorial en modelos de nivel RT).
- Con tantos puertos de entrada como señales diferentes aparezcan en la expresión.
- Con una funcionalidad especificada por los operadores que forman la expresión.



implementación de asignaciones concurrentes (ii)

- ⊗ Toda **asignación condicional de señal** (o su proceso equivalente) se implementa como un bloque de lógica combinacional:
- Con un único puerto de salida (que puede ser vectorial en modelos de nivel RT).
 - Con tantos puertos de entrada como señales diferentes aparezcan en el lado derecho de la asignación (independientemente de la expresión en la que ocurran).
 - Con un comportamiento que se corresponde con el de un multiplexor 2 a 1 cuyas 3 entradas están conectadas a las salidas de 3 bloques combinacionales. La funcionalidad de dichos bloques queda especificada por los operadores que forman cada una de las 3 expresiones de la sentencia.

`señal <= expresión when expresión_booleana else expresión;`

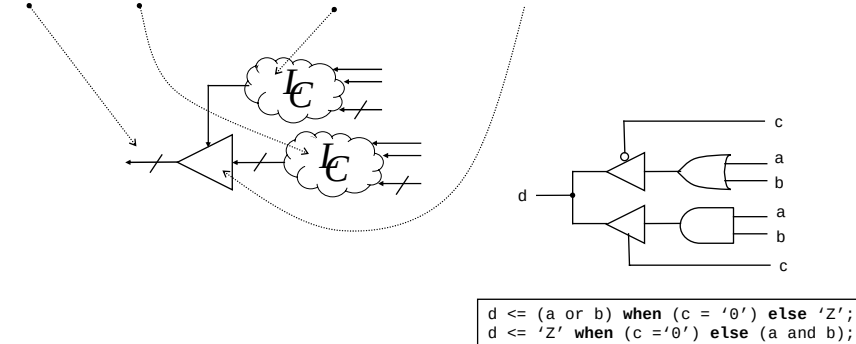


diseño automático de sistemas

implementación de asignaciones concurrentes (iii)

- ⊗ La **asignación condicional** del valor explícito 'Z' (uno de los valores del tipo *std_logic*) especifica la capacidad tri-estado del puerto de salida de la lógica combinacional especificada por el resto de la sentencia.

`señal <= expresión when expresión_booleana else (others=>'Z');`

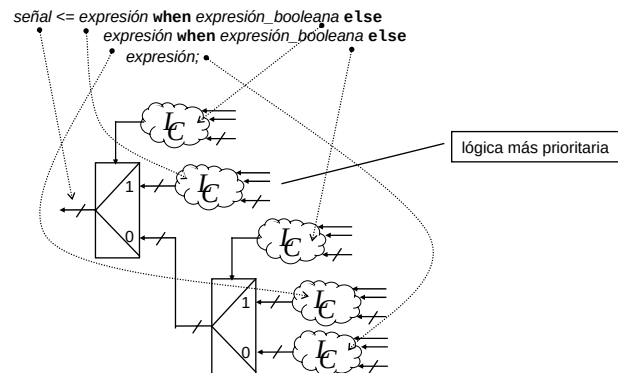


diseño automático de sistemas

implementación de asignaciones concurrentes (iv)

- ☒ Una colección de **asignaciones condicionales** anidadas (o su proceso equivalente) se implementada como lógica combinacional en cascada que establece prioridades explícitas entre varios cálculos.

© J.M. Mendias, 2004

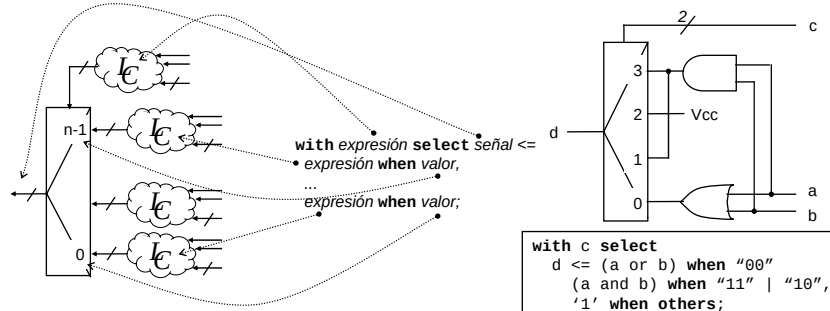


diseño automático de sistemas

implementación de asignaciones concurrentes (v)

- ☒ Toda **asignación selectiva de señal** (o su proceso equivalente) se implementa como un bloque de lógica combinacional:
- Con un único puerto de salida (que puede ser vectorial en modelos de nivel RT).
 - Con tantos puertos de entrada como señales diferentes aparezcan en el lado derecho de la asignación (independientemente de la expresión en la que ocurran).
 - Con un comportamiento que se corresponde con el de un multiplexor 2^n a 1 cuyas 2^n entradas están conectadas a las salidas de $n+1$ bloques combinacionales. La funcionalidad de dichos bloques queda especificada por los operadores que forman cada una de las $n+1$ expresiones de la sentencia. Siendo n el número de bits con los que se codifica el resultado de la expresión seleccionada.

© J.M. Mendias, 2004



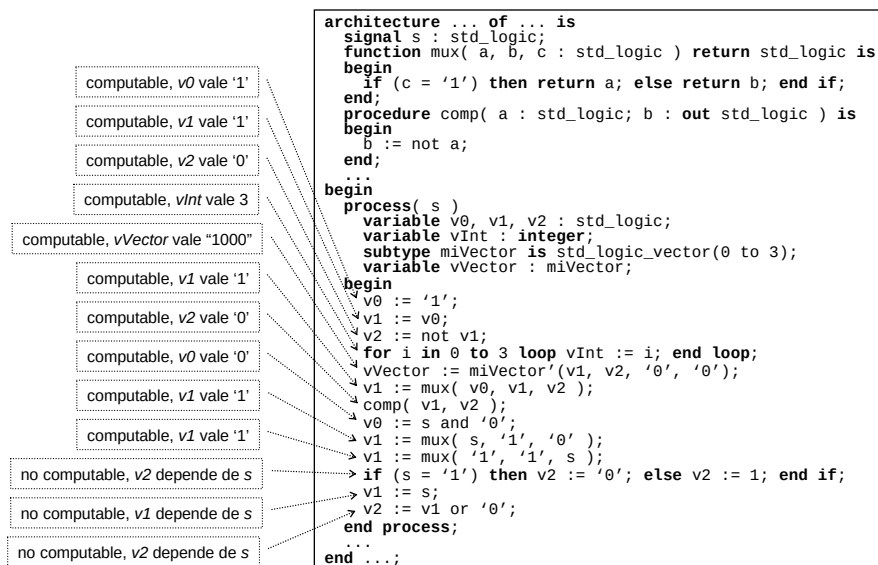
diseño automático de sistemas

implementación de expresiones (i)

- ☒ Las expresiones son el mecanismo fundamental para especificar la funcionalidad combinacional de un circuito.
- ☒ Las expresiones VHDL, desde el punto de vista de síntesis, pueden clasificarse en:
 - **Expresiones computables:** aquellas cuyo resultado puede ser determinado en tiempo de análisis, por tanto tienen un valor estático.
 - ✓ No requieren HW que las calcule, se implementan como conexiones a VCC y/o GND.
 - **Expresiones no computables:** aquellas cuyo resultado no puede ser determinado en tiempo de análisis, por tanto tienen un valor dinámico.
 - ✓ Se implementan mediante HW combinacional que las calcula.
- ☒ Son expresiones no computables aquellas que contengan:
 - Señales o puertos.
 - Variables que hayan sido anteriormente asignadas por una expresión no computable.
 - Variables que hayan sido anteriormente asignadas (aunque sea por expresiones computables) en función del valor de una condición no computable.

diseño automático de sistemas

implementación de expresiones (ii)

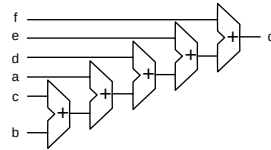
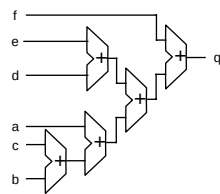


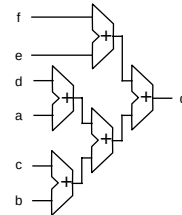
diseño automático de sistemas

implementación de expresiones (iii)

- ☒ Las expresiones simples (formadas un único operador) especifican una conducta primitiva sin estructura interna aparente.
- ☒ Cuando se especifica una conducta compleja se necesita utilizar una expresión compuesta (formada por varios operadores) o varias expresiones que se comunican a través de señales (o variables), por ello se está especificando:
 - La propia conducta a diseñar.
 - Una ordenación de los cálculos que se traduce en una estructura inicial sobre la que la herramienta comenzará a optimizar.
- ☒ Dicha estructura inicial es determinante (sobre todo a nivel RT) a la hora de obtener buenos resultados, ya que puede desde facilitar la búsqueda del diseño ideal hasta ocultarlo.

© J.M. Mendias, 2004

$$q \leftarrow (((b+c)+a)+(d+e))+f;$$


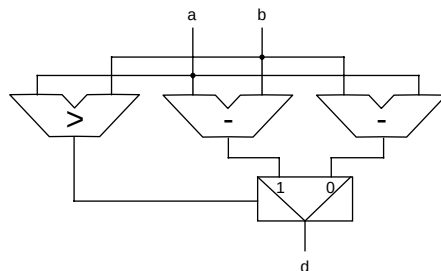
$$q \leftarrow b + c + a + d + e + f;$$


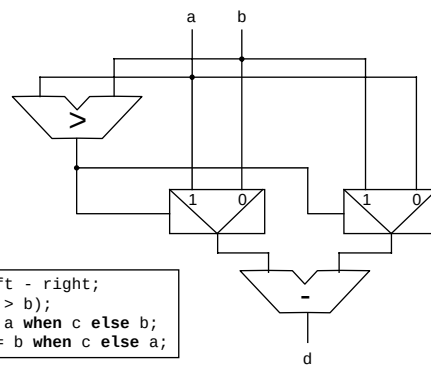
diseño automático de sistemas

• • • • •

implementación de expresiones (iv)

© J.M. Mendias, 2004



$$d \leftarrow (a - b) \text{ when } (a > b) \text{ else } (b - a);$$


$$\begin{aligned} d &\leftarrow \text{left} - \text{right}; \\ c &\leftarrow (a > b); \\ \text{left} &\leftarrow a \text{ when } c \text{ else } b; \\ \text{right} &\leftarrow b \text{ when } c \text{ else } a; \end{aligned}$$

diseño automático de sistemas

• • • • •

implementación de procesos (i)

- ☒ Un proceso especifica **HW combinacional** si todas las señales o variables escritas dentro del mismo se asignan al menos una vez bajo cualquier condición de ejecución.
- ☒ Por el contrario un proceso especifica **HW secuencial** si las señales o variables escritas dentro del mismo no se asignan por defecto, sólo bajo ciertas condiciones de ejecución
 - Típicamente tras la detección de un nivel o un evento en cierta señal
- ☒ Un proceso también especifica HW secuencial si contiene al menos una sentencia *wait*.

© J.M. Méndias, 2004

```

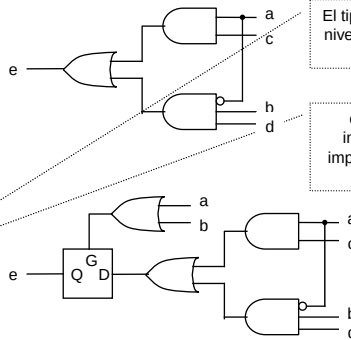
process( a, b, c, d )
begin
  if (a = '1') then
    e <= c;
  elsif (b = '1') then
    e <= d;
  else
    e <= '0';
  end if;
end process;

```

```

process( a, b, c, d )
begin
  if (a = '1') then
    e <= c;
  elsif (b = '1') then
    e <= d;
  end if;
end process;

```



El tipo de expresiones utilizadas (de nivel o de flanco) determinan el tipo de HW secuencial

cualquier sentencia *if* o *case* incompetentemente especificada implica la necesidad de almacenar los valores no modificados

diseño automático de sistemas

• • • • • • • •

implementación de procesos (ii): ejemplos

Latch tipo SR

```

process( set, rst )
begin
  if (set = '1') then
    q <= '1';
  elsif (rst = '1') then
    q <= '0';
  end if;
end process;

```

Latch tipo D

```

process( gate, d )
begin
  if (gate = '1') then
    q <= d;
  end if;
end process;

```

Latch tipo D con reset asíncrono

```

process( gate, set, rst, d )
begin
  if (rst = '1') then
    q <= '0';
  elsif (gate = '1') then
    q <= d;
  end if;
end process;

```

Latch tipo D con set y reset asíncrono

```

process( gate, set, rst, d )
begin
  if (set = '1') then
    q <= '1';
  elsif (rst = '1') then
    q <= '0';
  elsif (gate = '1') then
    q <= d;
  end if;
end process;

```

Latch tipo D master-slave

```

master: process( clk, d )
begin
  if (clk = '1') then
    q <= d;
  end if;
end process;
slave: process( clk, d )
begin
  if (clk = '0') then
    q <= d;
  end if;
end process;

```

© J.M. Méndias, 2004

diseño automático de sistemas

• • • • • • • •

implementación de procesos (iii): ejemplos

Flip-flop tipo D

```
process( clk )
begin
  if (clk'event and clk = '1') then
    q <= d;
  end if;
end process;
```

```
process
begin
  wait until (clk'event and clk = '1');
  q <= d;
end process;
```

Flip-flop tipo JK

```
process
  variable jk : std_logic_vector (1 downto 0);
begin
  if (clk'event and clk = '1') then
    jk := (j & k);
    case jk is
      when "01" => q <= '0';
      when "10" => q <= '1';
      when "11" => q <= not( q );
      when "00" => q <= q;
      when others => q <= 'X';
    end case;
  end if;
end process;
```

Flip-flop tipo T

```
process( clk )
begin
  if (clk'event and clk = '1') then
    q <= not( q );
  end if;
end process;
```

Flip-flop tipo D con reset síncrono

```
process( clk, rst )
begin
  if (clk'event and clk = '1') then
    if (rst = '1') then
      q <= '0';
    else
      q <= d;
    end if;
  end if;
end process;
```

Flip-flop tipo D con reset asíncrono

```
process( clk, rst )
begin
  if (rst = '1') then
    q <= '0';
  elsif (clk'event and clk = '1') then
    q <= d;
  end if;
end process;
```

diseño automático de sistemas

implementación de procesos (iv): limitaciones

☒ Al especificar elementos de memoria disparados por flanco existen algunas restricciones:

- Sólo existen 2 tipos de especificaciones de flanco válidas:
 - ✓ `clk'event and clk='1'` *flanco de subida*
 - ✓ `clk'event and clk='0'` *flanco de bajada*
- Una especificación de flanco no puede utilizarse como operando.
 - ✓ aunque su presencia en VHDL pueda evaluarse, un evento HW es el punto de división entre dos valores estables diferentes; por tanto, al no ser un valor en sí mismo, no puede utilizarse como argumento en un cálculo.
- Una sentencia *if* que tenga por condición una especificación de flanco no puede tener rama *else*.
 - ✓ en caso contrario la acción especificada debería realizarse en todo momento menos en el preciso instante en el que el reloj cambia.
- En sentencias *if-then-elsif* la especificación de flanco sólo podrá ser la condición del último *if* (que no podrá tener rama *else*).
- En un proceso solo puede existir una única especificación de flanco.
 - ✓ en caso contrario se estaría especificando HW secuencial sensible a varios relojes.
- Un variable asignada en el cuerpo de una sentencia *if-then* cuya condición sea una especificación de flanco, no puede ser usada fuera del mismo.
 - ✓ en caso contrario existiría una inconsistencia entre el comportamiento del circuito que se está especificado y el comportamiento que se simularía.

diseño automático de sistemas

implementación de procesos (v): limitaciones

© J.M. Méndias, 2004

```
process( rst, clk )
begin
  if (clk'event and clk='1' and rst='1') then
    ...
  end if;
end process;
```

una especificación de flanco no puede utilizarse como operando

```
process( clk )
begin
  if not(clk'event and clk='1') then
    ...
  end if;
end process;
```

en un proceso sólo puede existir una especificación de flanco

```
process( clk1, clk2 )
begin
  if (clk1'event and clk1='1') then
    ...
  end if;
  if (clk2'event and clk2='1') then
    ...
  end if;
end process;
```

```
process( clk )
begin
  if ... then
    ...
  elsif (clk'event and clk='1') then
    ...
  else
    ...
  end if;
end process;
```

un if con especificación de flanco no puede tener rama else

```
process( clk )
begin
  if (clk'event and clk='1') then
    ...
  else
    ...
  end if;
end process;
```

```
process( clk )
variable aux : ...;
begin
  if (clk'event and clk='1') then
    aux := ...;
    z <= aux;
  end if;
  z <= aux;
end process;
```

asignación válida

asignación no válida

diseño automático de sistemas

implementación de procesos (vi): variables vs. señales

© J.M. Méndias, 2004

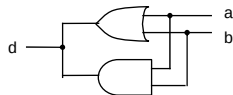
Las **señales** deben usarse:

- Para almacenar el estado de los procesos 'secuenciales'.
- Como medio de comunicación entre procesos u otras sentencias concurrentes.
 - ✓ El orden de asignación concurrente de señales no importa.
 - ✓ El orden de asignación secuencial de señales sí importa: si bajo unas condiciones de ejecución un proceso asigna varias veces una misma señal sólo se diseñará lógica para implementar la última asignación.
 - Esto es útil en procesos 'combinacionales', que pueden tener a su comienzo una asignación incondicional del valor por defecto de todas las señales y posteriormente asignaciones selectivas.

Las **variables** deben usarse

- Para simplificar las expresiones contenidas en los procesos 'combinacionales'. Reducir o preferiblemente eliminar su uso en procesos 'secuenciales'.
- ✓ El orden de asignación de variables puede importar si se asignan o no antes de ser usadas y si el proceso es 'combinacional' o 'secuencial'.

```
architecture ...
...
begin
  d <= a and b;
  d <= a or b;
  ...
end ...;
```

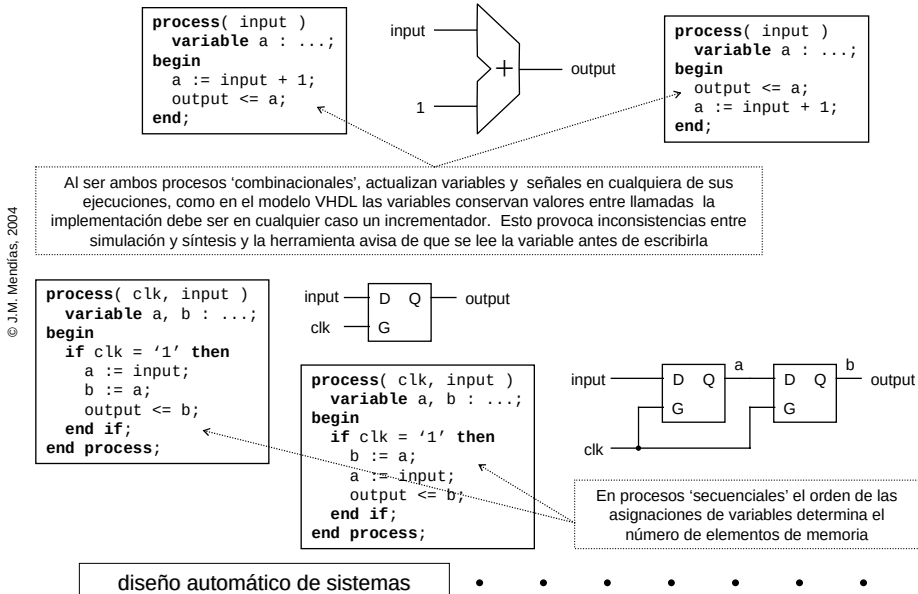


```
architecture ...
...
begin
  process( a, b )
    d <= a and b;
    d <= a or b;
  end process
  ...
end ...;
```



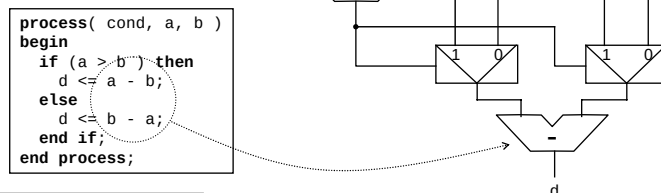
diseño automático de sistemas

implementación de procesos (vii): variables vs. señales



implementación de procesos (viii): reuso

- © J.M. Mendias, 2004
- ☒ Por defecto, cada ocurrencia de un operador da lugar a un bloque de lógica (recurso) que la implementa.
 - ☒ Cuando un único bloque de lógica puede implementar varias ocurrencias de un operador se dice que dichas ocurrencias **comparten el recurso**.
 - Esto es posible si la ejecución de dichas ocurrencias es **mutuamente exclusiva**, es decir, que no existe ninguna condición de ejecución bajo la cual se necesite calcular simultáneamente más de uno de los operadores que comparten el recurso.
 - ☒ Para que dos operadores compartan un recurso debe cumplirse que:
 - ocurran dentro del mismo proceso.
 - sean mutuamente exclusivos.
 - sean compatibles.
 - que la implementación del reuso no incurra en una realimentación combinacional.



implementación de procesos (ix): reuso

- ☒ Dos ocurrencias del mismo operador son siempre compatibles. Dos ocurrencias de diferentes operadores serán compatibles si ambas pertenecen a uno de los siguientes conjuntos
- { +, - }
 - { >, >=, <, <= }

© J.M. Mendias, 2004

```

process(a,b,c,d,e,f,g,h,i,j,k,l,m,n,c1,c2)
begin
  z1 <= a + b;
  if c1 then
    z2 <= c + d;
  else
    z2 <= e + f;
    if c2 then
      z3 <= g + h;
    else
      z3 <= i + j;
    end if;
  end if;
  if not c1 then
    z4 <= k + l;
  else
    z4 <= m + n;
  end if;
end process;

```

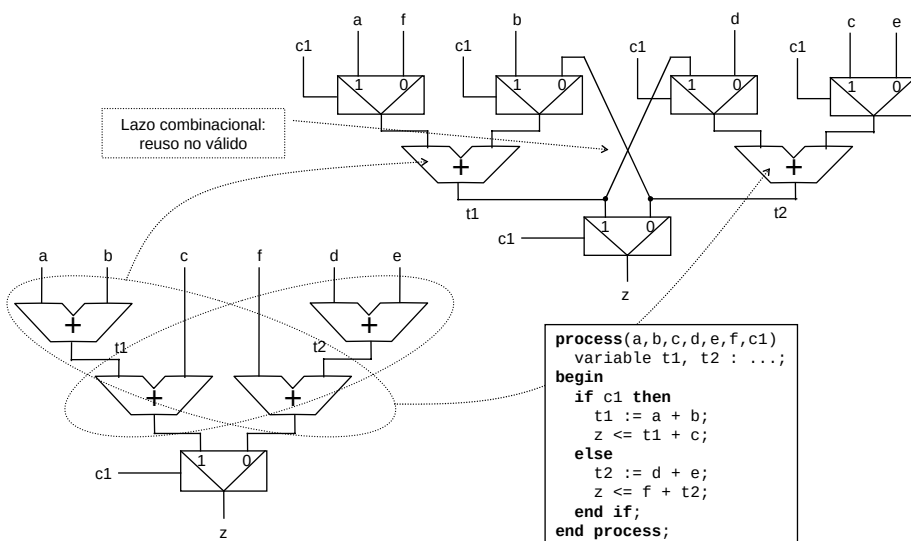
VHDL Compiler considera que todas las condiciones no tienen relación

	a + b	c + d	e + f	g + h	i + j	k + l	m + n
a + b		no	no	no	no	no	no
c + d	no		si	si	si	no	no
e + f	no	si		no	no	no	no
g + h	no	si	no		si	no	no
i + j	no	si	no	si		no	no
k + l	no	no	no	no	no		si
m + n	no	no	no	no	no	si	

diseño automático de sistemas

implementación de procesos (x): reuso

© J.M. Mendias, 2004



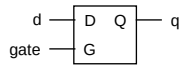
diseño automático de sistemas

implementación de subprogramas

- ☒ Los subprogramas **siempre** se implementa como un bloque de lógica combinacional
- Pro ello, los procedimientos no pueden conterner sentencias *wait*.

© J.M. Mendias, 2004

```
process( gate, d )
begin
  if (gate = '1') then
    q <= d;
  end if;
end process;
```



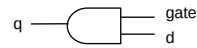
Las variables de subprogramas son dinámicas, por lo que no retienen valores entre llamadas

Las funciones siempre deben devolver un valor

Cuando una variable se crea se inicializa por defecto al primer valor de su declaración de tipo, en el caso de *bit*, este valor es '0' y se conservará hasta que se asigne otro

```
function fun( d, gate : bit ) return bit is
  variable q: bit;
begin
  if (gate = '1') then
    q := d;
  end if;
  return q;
end;
...
q <= fun( d, gate );
...
```

d	gate	return
0	0	0
0	1	0
1	0	0
1	1	1



diseño automático de sistemas

paquetes estándar (i): *std_logic_1164*

© J.M. Mendias, 2004

- ☒ El paquete **std_logic_1164** define 2 tipos atómicos multivaluados (uno resuelto y otro no) y 2 tipos vectoriales asociados:

```
type std_ulogic is ( 'U', 'X', '0', '1', 'Z', 'W', 'L', 'H', '-' );
type std_ulogic_vector is array ( natural range <> ) of std_ulogic;
subtype std_logic is resolved std_ulogic;
type std_logic_vector is array ( natural range <> ) of std_logic;
```

- ☒ El significado de cada uno de los 9 valores es:

- '0' y '1': valores lógicos fuertes
- 'L' y 'H': valores lógicos débiles
- 'Z': alta impedancia
- 'X': valor fuerte desconocido
- 'W': valor débil desconocido
- 'U': no inicializado
- '-': indiferencia (don't care)

- ☒ Dentro del paquete se definen también una colección de operadores y funciones lógicas sobre dichos tipos (tanto atómicos como vectoriales).

- **and, nand, or, nor, xor, xnor, not**

- ☒ Este paquete define un estándar para la especificación del interconexionado entre modelos VHDL (está soportado por todas las herramientas de síntesis).

- Los puertos de un diseño deberán ser siempre de estos tipos.
- También es recomendable que los objetos internos se declaren de estos tipos.

diseño automático de sistemas

paquetes estándar (ii): *std_logic_1164*

Interpretación de los valores del tipo

- ☒ Los valores lógicos fuertes, '0' y '1' especifican respectivamente conexiones a tierra o a alimentación.
- ☒ Los valores lógicos débiles, 'L' y 'H' especifican respectivamente conexiones a tierra o a alimentación.
- ☒ La utilización del valor metalógico 'Z' en una asignación, especifica un buffer triestado habilitado por la misma condición bajo la cual se realiza tal asignación. En cualquier otro caso, se interpreta como el resto de los valores metalógicos.
- ☒ La utilización del valor metalógico '-' en una asignación, especifica una indiferencia en el valor efectivo que debe asignarse. En cualquier otro caso, se interpreta como el resto de los valores metalógicos.
- ☒ Los valores metalógicos 'U', 'W', 'X' y en ocasiones '-' y 'Z', carecen de sentido en síntesis.
 - El resultado de una comparación de igualdad con un valor metalógico explícito es siempre falsa desde el punto de vista de síntesis, luego no requiere HW.
 - El resultado de una comparación de desigualdad con un valor metalógico explícito es siempre cierta desde el punto de vista de síntesis, luego no requiere HW.
 - Cuando se utiliza un valor metalógico explícito en una selección se considera, desde el punto de vista de síntesis, que la alternativa especificada no se ejecutará nunca, luego no requiere HW.
 - Está prohibida la utilización de un valor metalógico explícito como operando de un operador aritmético o lógico.

```
z <= '0' when s = "00-1" else '1'; z <= '1';
```

```
z <= '0' when s /= "00-1" else '1'; z <= '0';
```

```
z <= '0' when s = "0011" else '-'; z <= '0';
```

```
with s select z <=
'0' when "0000"
'1' when "0001"
x1  when "0--"
x2  when others;
```

```
with s select z <=
'0' when "0000"
'1' when "0001"
x2  when others;
```

diseño automático de sistemas

paquetes estándar (iii): *std_logic_arith*

- ☒ El paquete **std_logic_arith** define 2 tipos de datos vectoriales que, basados en el multivaluado *std_logic*, son interpretables como valores numéricos codificados. La longitud del vector determina el rango de valores representables por un objeto de tales tipos.
- ☒ El tipo **UNSIGNED** especifica un número natural representado en binario puro (estando el bit más significativo colocado a la izquierda).

```
type UNSIGNED is array (natural range <>) of std_logic;
```

- El número 8 puede representarse como **UNSIGNED'("1000")** o como **UNSIGNED'("001000")**

- ☒ El tipo **SIGNED** especifica un número entero representado en complemento a 2 (estando el bit de signo colocado a la izquierda).

```
type SIGNED is array (natural range <>) of std_logic;
```

- El número +5 puede representarse como **SIGNED'("0101")** o como **SIGNED'("00101")**
- El número -5 puede representarse como **SIGNED'("1011")** o como **SIGNED'("1111011")**

- ☒ Dentro del paquete existen:

- **Funciones de conversión** sobrecargadas entre los tipos *INTEGER*, *UNSIGNED*, *SIGNED*, *STD_ULONGIC* y *STD_LOGIC_VECTOR*.
- **Operadores aritméticos** para los tipos *INTEGER*, *UNSIGNED*, *SIGNED* y *STD_ULONGIC*.
- **Operadores relacionales** para los tipos *INTEGER*, *UNSIGNED* y *SIGNED*.
- **Funciones de desplazamiento aritmético** para los tipos *SIGNED* y *UNSIGNED*.

diseño automático de sistemas

paquetes estándar (iv): *std_logic_arith*

Perfiles de las funciones de conversión

```
subtype SMALL_INT is INTEGER range 0 to 1;

function CONV_UNSIGNED(ARG: INTEGER; SIZE: INTEGER) return UNSIGNED;
function CONV_UNSIGNED(ARG: UNSIGNED; SIZE: INTEGER) return UNSIGNED;
function CONV_UNSIGNED(ARG: SIGNED; SIZE: INTEGER) return UNSIGNED;
function CONV_UNSIGNED(ARG: STD_ULOGIC; SIZE: INTEGER) return UNSIGNED;

function CONV_SIGNED(ARG: INTEGER; SIZE: INTEGER) return SIGNED;
function CONV_SIGNED(ARG: UNSIGNED; SIZE: INTEGER) return SIGNED;
function CONV_SIGNED(ARG: SIGNED; SIZE: INTEGER) return SIGNED;
function CONV_SIGNED(ARG: STD_ULOGIC; SIZE: INTEGER) return SIGNED;

function CONV_STD_LOGIC_VECTOR(ARG: INTEGER; SIZE: INTEGER) return STD_LOGIC_VECTOR;
function CONV_STD_LOGIC_VECTOR(ARG: UNSIGNED; SIZE: INTEGER) return STD_LOGIC_VECTOR;
function CONV_STD_LOGIC_VECTOR(ARG: SIGNED; SIZE: INTEGER) return STD_LOGIC_VECTOR;
function CONV_STD_LOGIC_VECTOR(ARG: STD_ULOGIC; SIZE: INTEGER) return STD_LOGIC_VECTOR;

function CONV_INTEGER(ARG: INTEGER) return INTEGER;
function CONV_INTEGER(ARG: UNSIGNED) return INTEGER;
function CONV_INTEGER(ARG: SIGNED) return INTEGER;
function CONV_INTEGER(ARG: STD_ULOGIC) return SMALL_INT;
```

convierten el entero a una representación con una anchura dada: si el número de bits requeridos por el valor a convertir fuera menor que el especificado, la representación se extiende adecuadamente; si por el contrario fuera mayor, se trunca por la izquierda.

VHDL define que cualquier entero debe estar comprendido entre -2147483647 y 2147483647, un rango representable con un *UNSIGNED* de 31 bits o un *SIGNED* de 32 bits

diseño automático de sistemas

paquetes estándar (v): *std_logic_arith*

Perfiles de las funciones aritméticas

```
function "+"(L: UNSIGNED; R: UNSIGNED) return UNSIGNED;
function "+"(L: SIGNED; R: SIGNED) return SIGNED;
function "+"(L: UNSIGNED; R: SIGNED) return SIGNED;
function "+"(L: SIGNED; R: UNSIGNED) return SIGNED;
function "+"(L: UNSIGNED; R: INTEGER) return UNSIGNED;
function "+"(L: SIGNED; R: INTEGER) return SIGNED;
function "+"(L: INTEGER; R: SIGNED) return SIGNED;
function "+"(L: UNSIGNED; R: STD_ULOGIC) return UNSIGNED;
function "+"(L: STD_ULOGIC; R: UNSIGNED) return UNSIGNED;
function "+"(L: SIGNED; R: STD_ULOGIC) return SIGNED;
function "+"(L: STD_ULOGIC; R: SIGNED) return SIGNED;
function "+"(L: UNSIGNED; R: UNSIGNED) return STD_LOGIC_VECTOR;
function "+"(L: SIGNED; R: SIGNED) return STD_LOGIC_VECTOR;
function "+"(L: UNSIGNED; R: SIGNED) return STD_LOGIC_VECTOR;
function "+"(L: SIGNED; R: UNSIGNED) return STD_LOGIC_VECTOR;
function "+"(L: UNSIGNED; R: INTEGER) return STD_LOGIC_VECTOR;
function "+"(L: INTEGER; R: UNSIGNED) return STD_LOGIC_VECTOR;
function "+"(L: SIGNED; R: INTEGER) return STD_LOGIC_VECTOR;
function "+"(L: INTEGER; R: SIGNED) return STD_LOGIC_VECTOR;
function "+"(L: UNSIGNED; R: STD_ULOGIC) return STD_LOGIC_VECTOR;
function "+"(L: STD_ULOGIC; R: UNSIGNED) return STD_LOGIC_VECTOR;
function "+"(L: SIGNED; R: STD_ULOGIC) return STD_LOGIC_VECTOR;
function "+"(L: STD_ULOGIC; R: SIGNED) return STD_LOGIC_VECTOR;
```

Cuando los dos argumentos de la función son de tipo *UNSIGNED* o *SIGNED*, la anchura del valor devuelto es la mayor de las anchuras de los argumentos.

Cuando se suma o resta un *UNSIGNED* y un *SIGNED* que es de la misma anchura o menor, el valor devuelto es un bit más ancho que el argumento de tipo *UNSIGNED*

Cuando en la función existe un único argumento de tipo *UNSIGNED* o *SIGNED*, el valor devuelto tiene la misma anchura que dicho argumento.

diseño automático de sistemas

paquetes estándar (vi): *std_logic_arith*

Perfiles de las funciones aritméticas (cont.)

```
function ""(L: UNSIGNED; R: UNSIGNED) return UNSIGNED;
function ""(L: SIGNED; R: SIGNED) return SIGNED;
function ""(L: SIGNED; R: UNSIGNED) return SIGNED;
function ""(L: UNSIGNED; R: SIGNED) return SIGNED;

function ""(L: UNSIGNED; R: UNSIGNED) return STD_LOGIC_VECTOR;
function ""(L: SIGNED; R: SIGNED) return STD_LOGIC_VECTOR;
function ""(L: SIGNED; R: UNSIGNED) return STD_LOGIC_VECTOR;
function ""(L: UNSIGNED; R: SIGNED) return STD_LOGIC_VECTOR;

function "+"(L: UNSIGNED) return UNSIGNED;
function "+"(L: SIGNED) return SIGNED;
function "-"(L: SIGNED) return SIGNED;
function "ABS"(L: SIGNED) return SIGNED;

function "+"(L: UNSIGNED) return STD_LOGIC_VECTOR;
function "+"(L: SIGNED) return STD_LOGIC_VECTOR;
function "-"(L: SIGNED) return STD_LOGIC_VECTOR;
function "ABS"(L: SIGNED) return STD_LOGIC_VECTOR;
```

© J.M. Mendias, 2004

paquetes estándar (vii): *std_logic_arith*

Perfiles de las funciones relacionales

```
function "<"(L: UNSIGNED; R: UNSIGNED) return BOOLEAN;
function "<"(L: SIGNED; R: SIGNED) return BOOLEAN;
function "<"(L: UNSIGNED; R: SIGNED) return BOOLEAN;
function "<"(L: SIGNED; R: UNSIGNED) return BOOLEAN;
function "<"(L: UNSIGNED; R: INTEGER) return BOOLEAN;
function "<"(L: INTEGER; R: UNSIGNED) return BOOLEAN;
function "<"(L: SIGNED; R: INTEGER) return BOOLEAN;
function "<"(L: INTEGER; R: SIGNED) return BOOLEAN;
```

estas funciones comparan el valor numérico de los argumentos y no la particular configuración de bits

arg1	op	arg2	unsigned	signed	std_logic_vector
"000"	=	"000"	true	true	true
"00"	=	"000"	true	true	false
"100"	=	"0100"	true	false	false
"000"	<	"000"	false	false	false
"00"	<	"000"	false	false	true
"100"	<	"0100"	false	true	false

© J.M. Mendias, 2004
idénticos para <=, >, >=, =, /=

Perfiles de las funciones de desplazamiento

```
function SHL(ARG: UNSIGNED;COUNT: UNSIGNED) return UNSIGNED;
function SHL(ARG: SIGNED;COUNT: UNSIGNED) return SIGNED;
function SHR(ARG: UNSIGNED;COUNT: UNSIGNED) return UNSIGNED;
function SHR(ARG: SIGNED;COUNT: UNSIGNED) return SIGNED
```

paquetes estándar (viii): *std_logic_arith*

- ⊗ El uso de las funciones del paquete *std_logic_arith*, requiere que los argumentos vectoriales sean de tipo *signed* o *unsigned*. Si se desea usarlas con argumentos de tipo *std_logic_vector*, es necesario convertir explícitamente tales argumentos a uno de dichos tipos (para indicar la representación de los datos).
 - El paquete *std_logic_arith*, deberá usarse en aquellas especificaciones que manipulen tanto datos con signo como sin signo.
- ⊗ El paquete ***std_logic_unsigned*** define una nueva colección de funciones que permite la manipulación directa de objetos de tipo *std_logic_vector*, interpretándolos **siempre** como valores sin signo.
 - Deberá usarse en aquellas especificaciones que únicamente manipulen datos sin signo.
- ⊗ El paquete ***std_logic_unsigned*** define una nueva colección de funciones que permite la manipulación directa de objetos de tipo *std_logic_vector*, interpretándolos **siempre** como valores sin signo.
 - Deberá usarse en aquellas especificaciones que únicamente manipulen datos con signo.
- ⊗ Ambos paquetes convierten adecuadamente los argumentos y hacen llamada a la función de *std_logic_arith* necesaria.

diseño automático de sistemas

• • • • • • • •

paquetes estándar (ix): *std_logic_unsigned*

Perfiles de las funciones del paquete *std_logic_unsigned*

```
function CONV_INTEGER(ARG: STD_LOGIC_VECTOR) return INTEGER;
function "+"(L: STD_LOGIC_VECTOR; R: STD_LOGIC_VECTOR) return STD_LOGIC_VECTOR;
function "+"(L: STD_LOGIC_VECTOR; R: INTEGER) return STD_LOGIC_VECTOR;
function "+"(L: INTEGER; R: STD_LOGIC_VECTOR) return STD_LOGIC_VECTOR;
function "+"(L: STD_LOGIC_VECTOR; R: STD_LOGIC) return STD_LOGIC_VECTOR;
function "+"(L: STD_LOGIC; R: STD_LOGIC_VECTOR) return STD_LOGIC_VECTOR;
...
function "-"(L: STD_LOGIC; R: STD_LOGIC_VECTOR) return STD_LOGIC_VECTOR;
function "+"(L: STD_LOGIC_VECTOR) return STD_LOGIC_VECTOR;
function "*" (L: STD_LOGIC_VECTOR; R: STD_LOGIC_VECTOR) return STD_LOGIC_VECTOR;
...
function "<"(L: STD_LOGIC_VECTOR; R: STD_LOGIC_VECTOR) return BOOLEAN;
function "<"(L: STD_LOGIC_VECTOR; R: INTEGER) return BOOLEAN;
function "<"(L: INTEGER; R: STD_LOGIC_VECTOR) return BOOLEAN;
...
function SHL(ARG:STD_LOGIC_VECTOR; COUNT: STD_LOGIC_VECTOR) return STD_LOGIC_VECTOR;
function SHR(ARG:STD_LOGIC_VECTOR; COUNT: STD_LOGIC_VECTOR) return STD_LOGIC_VECTOR;
```

diseño automático de sistemas

• • • • • • • •

paquetes estándar (x): *std_logic_signed*

Perfiles de las funciones del paquete *std_logic_signed*

```
function CONV_INTEGER(ARG: STD_LOGIC_VECTOR) return INTEGER;
function "+"(L: STD_LOGIC_VECTOR; R: STD_LOGIC_VECTOR) return STD_LOGIC_VECTOR;
function "+"(L: STD_LOGIC_VECTOR; R: INTEGER) return STD_LOGIC_VECTOR;
function "+"(L: INTEGER; R: STD_LOGIC_VECTOR) return STD_LOGIC_VECTOR;
function "+"(L: STD_LOGIC_VECTOR; R: STD_LOGIC) return STD_LOGIC_VECTOR;
function "+"(L: STD_LOGIC; R: STD_LOGIC_VECTOR) return STD_LOGIC_VECTOR;
...
function "-"(L: STD_LOGIC_VECTOR) return STD_LOGIC_VECTOR;
function "-"(L: STD_LOGIC_VECTOR) return STD_LOGIC_VECTOR;
function "ABS"(L: STD_LOGIC_VECTOR) return STD_LOGIC_VECTOR;
function "*" (L: STD_LOGIC_VECTOR; R: STD_LOGIC_VECTOR) return STD_LOGIC_VECTOR;
...
function "<"(L: STD_LOGIC_VECTOR; R: STD_LOGIC_VECTOR) return BOOLEAN;
function "<"(L: STD_LOGIC_VECTOR; R: INTEGER) return BOOLEAN;
function "<"(L: INTEGER; R: STD_LOGIC_VECTOR) return BOOLEAN;
...
function SHL(ARG:STD_LOGIC_VECTOR;COUNT: STD_LOGIC_VECTOR) return STD_LOGIC_VECTOR;
function SHR(ARG:STD_LOGIC_VECTOR;COUNT: STD_LOGIC_VECTOR) return STD_LOGIC_VECTOR;
```

diseño automático de sistemas

• • • • • • • •

especificación de HW usando VHDL: método

- ☒ Pensar siempre en **HW síncrono de nivel RT** (las herramientas que utilizaremos comienzan desde este nivel de abstracción).
 - No comenzar a codificar hasta tener el diagrama de bloques a nivel RT (módulos combinacionales y registros).
 - ✓ El uso de VHDL no evita el diseño manual de la estructura RT, lo que evita es el diseño lógico de los bloques combinacionales y la proyección tecnológica.
 - Cada bloque RT habitualmente deberá codificarse con una sentencia concurrente.
 - Se permiten especificaciones más abstractas, pero para que puedan obtenerse soluciones válidas es necesaria una codificación muy cuidadosa.
 - Nunca mezclar en un mismo diseño diferentes estilos de temporización:
 - ✓ No utilizar más de un reloj por diseño.
 - ✓ No mezclar diseños con temporización a nivel con temporización a flanco.
 - ✓ Diseñar a mano los bloques asíncronos.
- ☒ El núcleo fundamental de la lógica a sintetizar se especificará mediante expresiones.
- ☒ El resto de construcciones del lenguaje se utilizarán para estructurar el diseño y para especificar su temporización.

diseño automático de sistemas

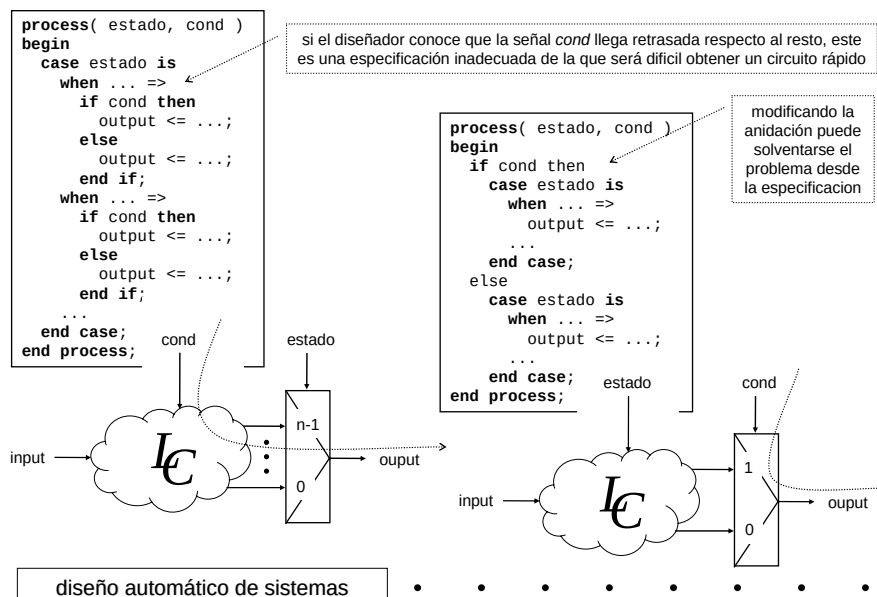
• • • • • • • •

estilos de especificación (i)

- ☒ Un circuito diseñado a nivel lógico se caracteriza por 3 aspectos:
 - La función que realiza.
 - La estructura interna que tiene.
 - Las componentes de biblioteca que utiliza.
- ☒ Para especificar con un mayor o menor nivel de detalle el circuito que se desea que la herramienta de síntesis obtenga, existen 3 estilos de uso del VHDL diferentes:
 - El que permite la **inferencia** de una estructura: se especifica únicamente el comportamiento del circuito del modo más abstracto posible dando libertad a la herramienta para que elija la estructura y componentes más adecuadas bajo ciertas condiciones de funcionamiento.
 - El que **implica** cierta estructura: se especifica tanto el comportamiento del circuito como una estructura global del mismo en base al conocimiento que tiene el diseñador de las condiciones de funcionamiento del circuito. La herramienta seleccionará las componentes del circuito
 - Que **instancia** ciertas componentes: se describe todo: función, estructura y componentes. La herramienta realiza chequeos.
- ☒ Los dos primeros estilos son independientes de tecnología, el tercero no.
- ☒ Deberá utilizarse preferiblemente el primer estilo, el segundo se reserva para pocas ocasiones. El tercero no es recomendable (es una captura de esquemas textual).
- ☒ Es también muy común que una descripción no siga un estilo puro y utilice:
 - **inferencia**: para especificar las partes no críticas del circuito
 - **implicación**: para especificar las partes críticas del circuito.
 - **instanciación**: para especificar elementos cruciales o que no puedan ser diseñados (asíncronos)

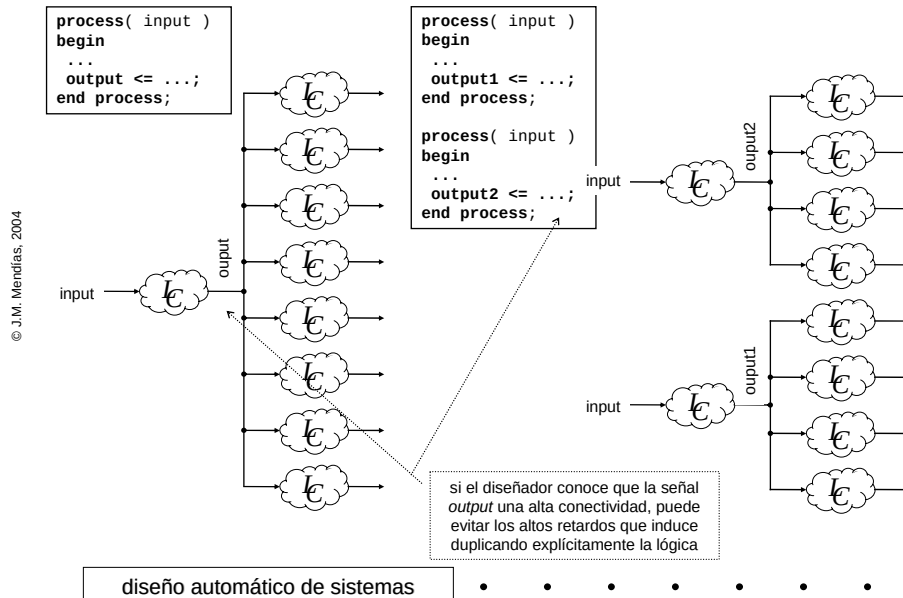
diseño automático de sistemas

estilos de especificación (ii): ejemplos



diseño automático de sistemas

estilos de especificación (iii): ejemplos



especificación de HW combinacional

- © J.M. Mendias, 2004
- ☒ La **lógica combinacional** puede ser especificada mediante:
 - **Asignaciones concurrentes** a señal.
 - **Procesos** sin sentencias *wait* tales que:
 - ✓ Todas las señales o variables escritas dentro del proceso sean asignadas al menos una vez en toda activación del mismo.
 - Si en el modelo existe un camino lógico tal que una señal o variable no se asigna, quiere decir que el valor del objeto debe ser retenido y, por tanto, que debe existir un elemento secuencial que lo memorice.
 - ✓ Para asegurar la coherencia entre simulación y síntesis, la **lista de sensibilidad debe ser completa**, es decir, que debe estar formada por todas las señales leídas dentro del proceso.
 - **Subprogramas.**
 - ✓ Especifican lógica combinacional siempre.
 - Independientemente de que se les llame en entornos secuenciales o concurrentes.
 - Independientemente de cómo estén codificados internamente.
 - ☒ Para asegurar que el diseño especificado es síncrono:
 - Ningún conjunto de señales asignadas por construcciones que especifiquen lógica combinacional pueden formar un lazo, es decir, una señal 'combinacional' no puede formar parte de la expresión que la define, en caso contrario se sintetizaría lógica asíncrona.

especificación de HW secuencial (i)

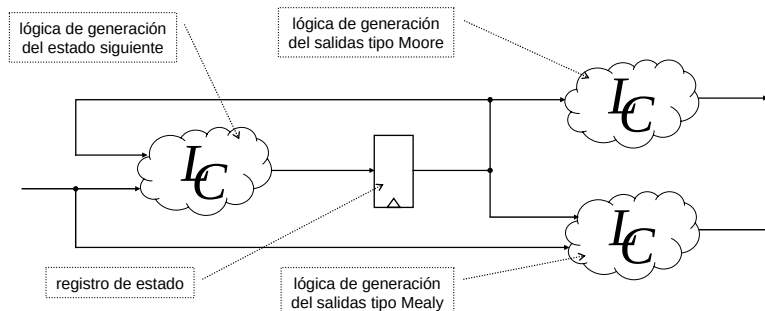
☒ La **lógica secuencial** se especifica mediante:

- **Procesos** que contengan una o varias sentencias *wait*.
 - ✓ Las únicas clases de sentencias *wait* permitidas son:
 - `wait until CLK = '1';`
 - `wait until CLK'event and CLK = '1';`
 - `wait until not CLK'stable and CLK = '1';`
 - ✓ Sólo puede especificarse lógica secuencial con temporización disparada por flanco.
 - ✓ Se especifican elementos de almacenamiento para:
 - Toda señal escrita por el proceso.
 - Toda variable que en alguna activación sea leída antes que escrita.
- **Procesos** que no contengan *wait*, tales que:
 - ✓ Existan algunas señales o variables escritas por el proceso que no se asignen bajo alguna condición de ejecución del mismo.
 - Dichas señales especificarán elementos de almacenamiento
 - Las condiciones que regulan su actualización determinarán la clase de elemento secuencial.
 - ✓ Puede especificarse cualquier lógica secuencial con temporización de cualquier tipo.
 - ✓ Para asegurar la coherencia entre simulación y síntesis, la **lista de sensibilidad debe ser parcial**, estando formada por todas aquellas señales que disparan cambios de estado (i.e: reloj, *reset*, *clear*, etc ...).

diseño automático de sistemas

especificación de HW secuencial (ii): método

- ☒ Según el **modelo de Huffman**, cualquier sistema secuencial puede considerarse formado por 2 únicos elementos:
- Un único registro de estado (que agrupa todos los elementos de memoria primitivos del sistema).
 - Una red combinacional que realiza los cálculos.
- ☒ Ampliando dicho modelo puede obtenerse un método fiable de especificación de sistemas secuenciales en VHDL. Todo sistema secuencial lo consideraremos formado por 4 elementos:
- Un registro de estado.
 - Una red combinacional que calcula el estado siguiente.
 - Una red combinacional que calcula las salidas que sólo dependen del estado (tipo Moore).
 - Una red combinacional que calcula las salidas que dependen del estado y de la entrada (tipo Mealy).



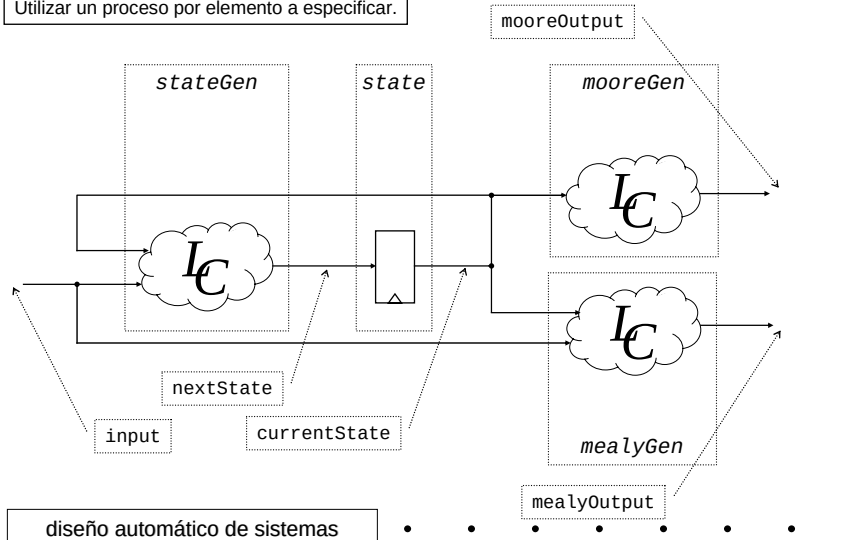
diseño automático de sistemas

especificación de HW secuencial (iii): método

primera alternativa

Utilizar un proceso por elemento a especificar.

© J.M. Mendias, 2004



especificación de HW secuencial (iv): método

valor por defecto de las señales asignadas por el proceso, asegura que se implemente como combinacional

© J.M. Mendias, 2004

```
stateGen:
process( currentState, input )
begin
  nextState <= currentState;
  case currentState is
    when ... =>
      if (input ...) then
        nextState <= ...;
      elsif (input ...) then
        ...
      else
        ...
      end if;
    ...
  end case;
end process;
```

```
state:
process( clk, rst )
begin
  if (rst = '1') then
    currentState <= ...;
  elsif (clk'event and clk='1') then
    currentState <= nextState;
  end if;
end process;
```

```
mealyGen:
process( currentState, input )
begin
  mealyOutput <= ...;
  case currentState is
    when ... =>
      if (input ...) then
        mealyOutput <= ...;
      elsif (input ...) then
        ...
      else
        ...
      end if;
    ...
  end case;
end process;
```

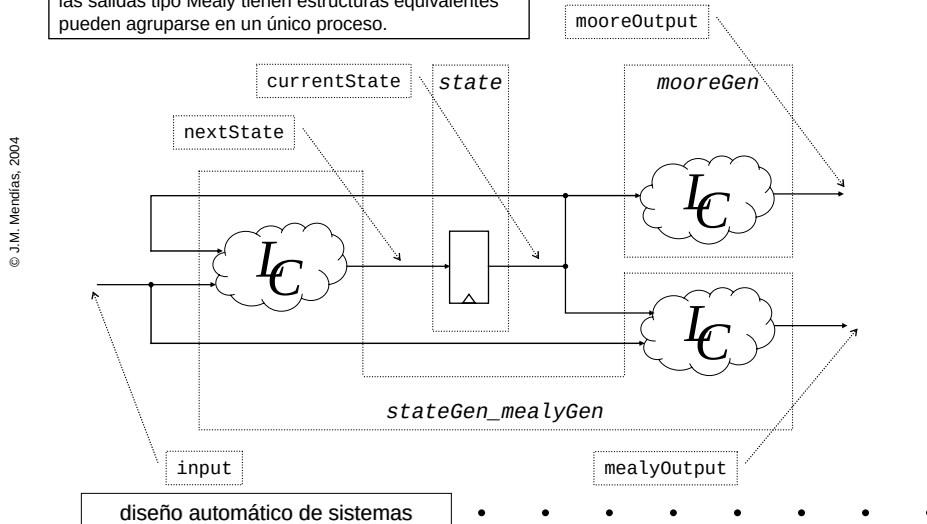
```
mooreGen:
process( currentState )
begin
  mooreOutput <= ...;
  case currentState is
    when ... =>
      mooreOutput <= ...
    ...
  end case;
end;
```

diseño automático de sistemas

especificación de HW secuencial (v): método

segunda alternativa

Dado que el cálculo del estado siguiente y el cálculo de las salidas tipo Mealy tienen estructuras equivalentes pueden agruparse en un único proceso.



especificación de HW secuencial (vi): método

© J.M. Mendias, 2004

```

stateGen_mealyGen:
process( currentState, input )
begin
  nextState <= currentState;
  mealyOutput <= ...;
  case currentState is
    when ... =>
      if (input ...) then
        nextState <= ...;
        mealyOutput <= ...;
      elsif (input ...) then
        ...
      else
        ...
      end if;
    ...
  end case;
end process;

```

```

state:
process( clk, rst )
begin
  if (rst = '1') then
    currentState <= ...;
  elsif (clk'event and clk='1') then
    currentState <= nextState;
  end if;
end process;

```

```

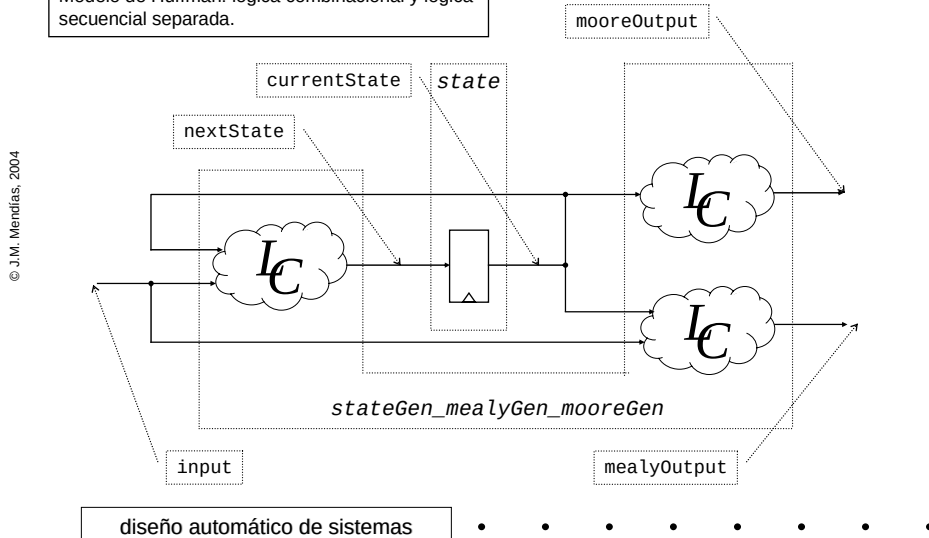
mooreGen:
process( currentState )
begin
  mooreOutput <= ...;
  case currentState is
    when ... =>
      mooreOutput <= ...;
    ...
  end case;
end;

```

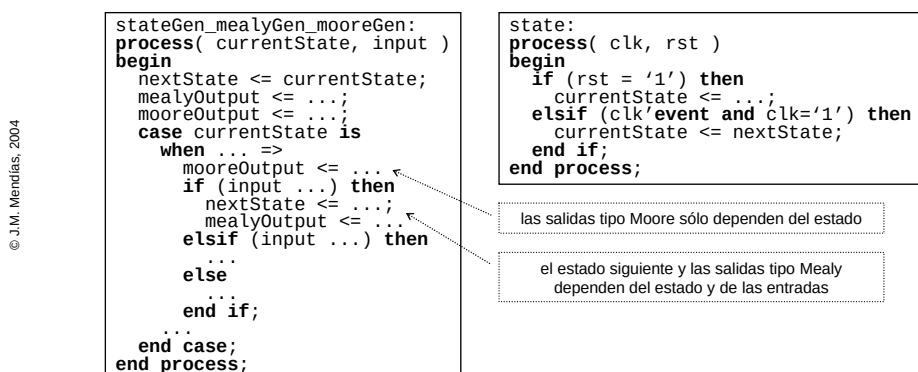
especificación de HW secuencial (vii): método

tercera alternativa

Modelo de Huffman: lógica combinacional y lógica secuencial separada.



especificación de HW secuencial (viii): método

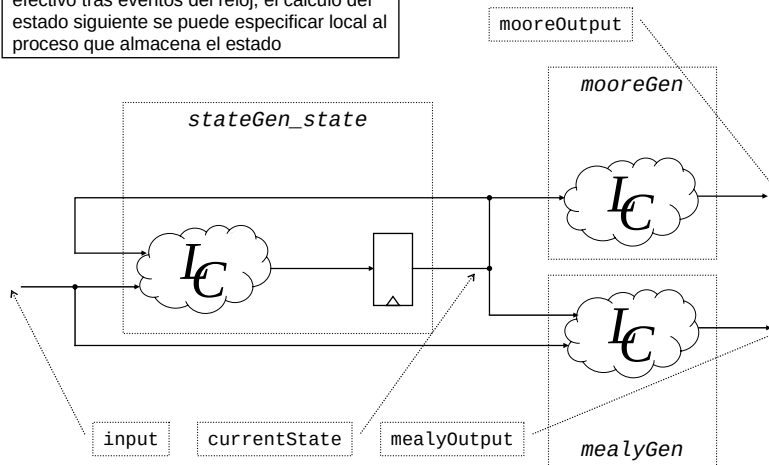


especificación de HW secuencial (ix): método

cuarta alternativa

Dado que el cambio de estado solo se hace efectivo tras eventos del reloj, el cálculo del estado siguiente se puede especificar local al proceso que almacena el estado

© J.M. Mendias, 2004



diseño automático de sistemas

• • • • •

especificación de HW secuencial (x): método

© J.M. Mendias, 2004

```
stateGen_state:
process( clk, rst, input )
begin
  if (rst = '1') then
    currentState <= ...;
  elsif (clk'event and clk='1') then
    currentState <= ...;
    case currentState is
      when ... =>
        if (input ...) then
          currentState <= ...;
        elsif (input ...) then
          ...
        else
          ...
        end if;
      ...
    end case;
  end if;
end process;
```

```
mooreGen:
process( currentState )
begin
  mooreOutput <= ...;
  case currentState is
    when ... =>
      mooreOutput <= ...
    ...
  end case;
end;
```

```
mealyGen:
process( currentState, input )
begin
  mealyOutput <= ...;
  case currentState is
    when ... =>
      if (input ...) then
        mealyOutput <= ...;
      elsif (input ...) then
        ...
      else
        ...
      end if;
    ...
  end case;
end process;
```

podría suprimirse

estado al que se salta por defecto (puede ser sobrescrito por posteriores asignaciones)

el cálculo del nuevo estado actual en función del estado actual (*currentState* es una señal) debe formar parte de la rama *then* de la especificación de flanco

diseño automático de sistemas

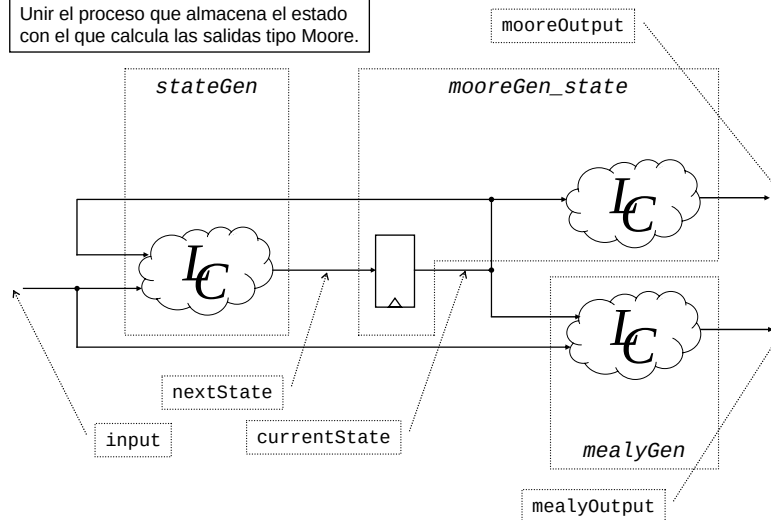
• • • • •

especificación de HW secuencial (xi): método

cuarta alternativa

Unir el proceso que almacena el estado con el que calcula las salidas tipo Moore.

© J.M. Méndias, 2004



diseño automático de sistemas

especificación de HW secuencial (xii): método

© J.M. Méndias, 2004

```
mooreGen_state:
process( clk, rst, currentState )
begin
    mooreOutput <= ...;
    case currentState is
        when ... =>
            mooreOutput <= ...
    end case;
    if (rst = '1') then
        currentState <= ...;
    elsif (clk'event and clk='1') then
        currentState <= nextState;
    end if;
end process;
```

```
mealyGen:
process( currentState, input )
begin
    mealyOutput <= ...;
    case currentState is
        when ... =>
            if (input ...) then
                mealyOutput <= ...;
            elsif (input ...) then
                ...
            end if;
    end case;
end process;
```

```
stateGen:
process( currentState, input )
begin
    nextState <= currentState;
    case currentState is
        when ... =>
            if (input ...) then
                nextState <= ...;
            elsif (input ...) then
                ...
            end if;
    end case;
end process;
```

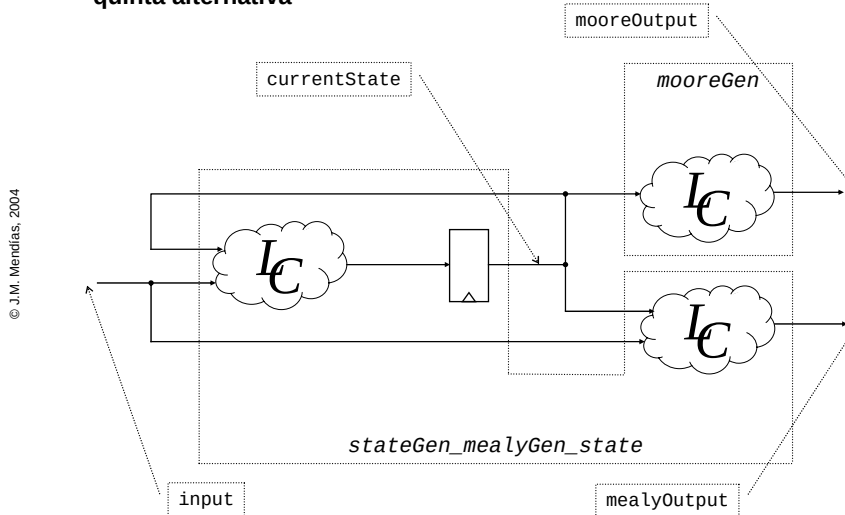
necesaria para que simule correctamente

el cálculo de la salida se realiza incondicionalmente, si se anidara dentro del *if*, se interpretaría que dichas salidas hay también que almacenarlas (aunque sólo cambien tras el cambio de estado)

diseño automático de sistemas

especificación de HW secuencial (xiii): método

quinta alternativa



diseño automático de sistemas

especificación de HW secuencial (xiv): método

```
mooreGen:
process( currentState )
begin
mooreOutput <= ...;
case currentState is
when ... =>
mooreOutput <= ...
end case;
end;
```

el cálculo de la salida se realiza incondicionalmente,
(ahora puede cambiar aunque no cambie el estado)

necesaria para que simule correctamente

necesaria para que simule correctamente

el cálculo del nuevo estado actual en función del estado
actual forma parte de la rama *then* de la especificación
de flanco

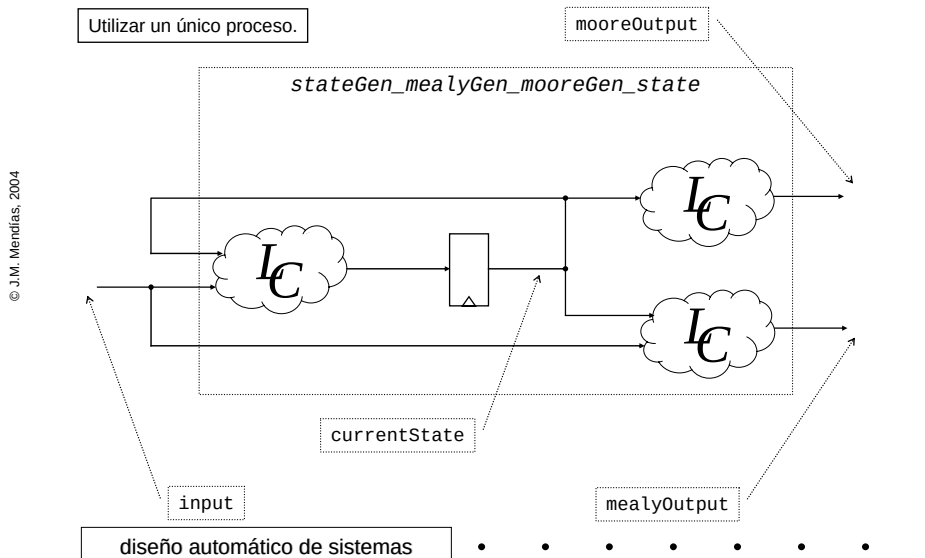
```
stateGen_MealyGen_state:
process( clk, rst, currentState, input )
begin
mealyOutput <= ...;
case currentState is
when ... =>
if (input ...) then
mealyOutput <= ...
elsif (input ...) then
...
end if;
...
end case;
if (rst = '1') then
currentState <= ...;
elsif (clk'event and clk='1') then
currentState <= ...;
case currentState is
when ... =>
if (input ...) then
currentState <= ...;
elsif (input ...) then
...
end if;
...
end case;
end if;
end process;
```

diseño automático de sistemas

especificación de HW secuencial (xv): método

sexta alternativa

Utilizar un único proceso.



especificación de HW secuencial (xvi): método

© J.M. Mendiás, 2004

el cálculo de la salida se realiza incondicionalmente

el calculo del nuevo estado actual en función del estado actual forma parte de la rama *then* de la especificación de flanco

Aunque resulte poco intuitivo, este es el método fiable de especificación de HW a nivel RT más abstracto, existen otros pero son difíciles de controlar

```
stateGen_mealyGen_mooreState_state:
process (clk, rst, currentState, input )
begin
    mealyOutput <= ...;
    mooreOutput <= ...;
    case currentState is
        when ... =>
            mooreOutput <= ...
            if (input ...) then
                mealyOutput <= ...
            elsif (input ...) then
                ...
            else
                ...
            end if;
        ...
    end case;
    if (rst = '1') then
        currentState <= ...;
    elsif (clk'event and clk='1') then
        currentState <= ...;
        case currentState is
            when ... =>
                if (input ...) then
                    currentState <= ...;
                elsif (input ...) then
                    ...
                else
                    ...
                end if;
            ...
        end case;
    end if;
end process;
```

multiplexor vectorial 2 a 1

```

library IEEE; use IEEE.std_logic_1164.all;

entity multiplexer is
  port(
    x0 : in std_logic_vector( 7 downto 0 );
    x1 : in std_logic_vector( 7 downto 0 );
    sel : in std_logic;
    y : out std_logic_vector( 7 downto 0 );
  end multiplexer;

  architecture syn1 of multiplexer is
  begin
    y <= x1 when (sel = '1') else x0;
  end syn;

  architecture syn2 of multiplexer is
  begin
    process( sel, x0, x1 )
    begin
      if sel='1' then
        y <= x1;
      else
        y <= x0;
      end if;
    end process;
  end syn3;

```

implica una estructura
de nivel lógico

```

architecture syn3 of multiplexer is
  signal aux : std_logic_vector( 7 downto 0 );
begin
  aux <= (others=>sel);
  y <= (x1 and aux) or (x0 and not aux);
end syn2;

```

diseño automático de sistemas

• • • • • • • •

multiplexor vectorial genérico 2 a 1

```

library IEEE; use IEEE.std_logic_1164.all;

entity multiplexer is
  generic( n : integer := 8 );
  port(
    x0 : in std_logic_vector( n-1 downto 0 );
    x1 : in std_logic_vector( n-1 downto 0 );
    sel : in std_logic;
    y : out std_logic_vector( n-1 downto 0 );
  end multiplexer;

  architecture syn of multiplexer is
  begin
    process( sel, x0, x1 )
    begin
      if sel='1' then
        y <= x1;
      else
        y <= x0;
      end if;
    end process;
  end syn3;

```

diseño automático de sistemas

• • • • • • • •

codificador de prioridad 8 a 3 (i)

```

library IEEE; use IEEE.std_logic_1164.all;

entity priorityEncoder is
  port(
    x : in std_logic_vector( 7 downto 0 );
    y : out std_logic_vector( 2 downto 0 );
    gs : out std_logic );
end priorityEncoder;

architecture syn of priorityEncoder is
begin
  process( x )
  begin
    if x(7)='1' then y <= "111"; gs <= '1';
    elsif x(6)='1' then y <= "110"; gs <= '1';
    elsif x(5)='1' then y <= "101"; gs <= '1';
    elsif x(4)='1' then y <= "100"; gs <= '1';
    elsif x(3)='1' then y <= "011"; gs <= '1';
    elsif x(2)='1' then y <= "010"; gs <= '1';
    elsif x(1)='1' then y <= "001"; gs <= '1';
    elsif x(0)='1' then y <= "000"; gs <= '1';
    else y <= "000"; gs <= '0';
    end if;
  end process;
end syn;

```

el proceso se activa cuando hay un evento en cualquiera de las componentes de x

x(7) se chequea en primer lugar es el más prioritario

la anidación de if especifica la prioridad de las entradas

la respuesta cuando no hay entradas activadas se especifica en la última rama else

diseño automático de sistemas

codificador de prioridad 8 a 3 (ii)

```

library IEEE; use IEEE.std_logic_1164.all;

entity priorityEncoder is
  port(
    x : in std_logic( 7 downto 0 );
    y : out std_logic_vector( 2 downto 0 );
    gs : out std_logic );
end priorityEncoder;

architecture syn of priorityEncoder is
begin
  process( x )
  begin
    y <= "000"; gs <= '0';
    if x(0)='1' then y <= "000"; gs <= '1'; end if;
    if x(1)='1' then y <= "001"; gs <= '1'; end if;
    if x(2)='1' then y <= "010"; gs <= '1'; end if;
    if x(3)='1' then y <= "011"; gs <= '1'; end if;
    if x(4)='1' then y <= "100"; gs <= '1'; end if;
    if x(5)='1' then y <= "101"; gs <= '1'; end if;
    if x(6)='1' then y <= "110"; gs <= '1'; end if;
    if x(7)='1' then y <= "111"; gs <= '1'; end if;
  end process;
end syn;

```

la prioridad de las entradas se especifica sobrescribiendo selectivamente el valor asignado por anteriores sentencias

valores 'por defecto' de las salidas cuando no hay entradas activadas

x(7) se chequea en último lugar es el más prioritario

diseño automático de sistemas

codificador de prioridad genérico

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_arith.all;

entity priorityEncoder is
  generic( n : integer := 3 );
  port(
    x : in std_logic( 2**n-1 downto 0 );
    y : out std_logic_vector( n-1 downto 0 );
    gs : out std_logic );
end priorityEncoder;

architecture syn of priorityEncoder is
begin
  process( x )
  begin
    y <= (others=>'0'); gs <= '0';
    for i in x'reverse_range loop
      if x(i)='1' then
        y <= conv_std_logic_vector( i, n );
        gs <= '1';
      end if;
    end loop;
  end process;
end syn;

```

la prioridad de las entradas se especifica sobrescribiendo selectivamente el valor asignado por anteriores sentencias

el rango de i es computable y el bucle puede desenrollarse

expresión computable

presente en paquete *std_logic_arith*

diseño automático de sistemas

• • • • •

decodificador 3 a 8 (i)

```

library IEEE; use IEEE.std_logic_1164.all;

entity decoder is
  port(
    x : in std_logic_vector( 2 downto 0 );
    en : in std_logic;
    y : out std_logic( 7 downto 0 ) );
end decoder;

architecture syn of decoder is
begin
  process( x, en )
  begin
    y <= "00000000";
    if en='1' then
      case x is
        when "000" => y(0) <= '1';
        when "001" => y(1) <= '1';
        when "010" => y(2) <= '1';
        when "011" => y(3) <= '1';
        when "100" => y(4) <= '1';
        when "101" => y(5) <= '1';
        when "110" => y(6) <= '1';
        when "111" => y(7) <= '1';
      end case;
    end if;
  end process;
end syn;

```

la selección se anida dentro de la habilitación define un reuso de los cálculos intermedios e implica una estructura en cascada

diseño automático de sistemas

• • • • •

decodificador 3 a 8 (ii)

```

library IEEE; use IEEE.std_logic_1164.all;

entity decoder is
  port(
    x : in std_logic_vector( 2 downto 0 );
    en : in std_logic;
    y : out std_logic( 7 downto 0 ) );
end decoder;

architecture syn of decoder is
begin
  y(0) <= '1' when (en='1' and x="000") else '0';
  y(1) <= '1' when (en='1' and x="001") else '0';
  y(2) <= '1' when (en='1' and x="010") else '0';
  y(3) <= '1' when (en='1' and x="011") else '0';
  y(4) <= '1' when (en='1' and x="100") else '0';
  y(5) <= '1' when (en='1' and x="101") else '0';
  y(6) <= '1' when (en='1' and x="110") else '0';
  y(7) <= '1' when (en='1' and x="111") else '0';
end syn;

```

cada salida se asigna por separado

no se comparte la expresión de habilitación
luego implica una estructura paralela

diseño automático de sistemas

decodificador generico

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_arith.all;

entity decoder is
  generic( n : integer := 3 );
  port(
    x : in std_logic_vector( n-1 downto 0 );
    en : in std_logic;
    y : out std_logic( 2**n-1 downto 0 ) );
end decoder;

architecture syn of decoder is
begin
  process( x, en )
  begin
    y <= (others=>'0');
    if en='1' then
      y( conv_integer( unsigned(x) ) ) <= '1';
    end if;
  end process;
end syn;

```

las conversiones de tipo no requieren HW
ya que sólo indican cómo debe interpretarse z

presente en paquete *std_logic_arith*

diseño automático de sistemas

sumador genérico (i)

```
library IEEE; use IEEE.std_logic_1164.all;

entity adder is
  generic( n : integer := 8 );
  port(
    x, y : in std_logic( n-1 downto 0 );
    cin : in std_logic;
    s : out std_logic_vector( n-1 downto 0 );
    cout : out std_logic );
end adder;

architecture syn of adder is
  signal c : std_logic_vector( n downto 0 );
begin
  c(0) <= cin;
  for i in s'reverse_range generate
    s(i) <= x(i) xor y(i) xor c(i);
    c(i+1) <= (x(i) and y(i)) or ((x(i) xor y(i)) and c(i));
  end generate;
  cout <= c(n);
end syn;
```

colección de señales intermedias para
conectar los acarreo intermedios

comienza a calcular por el bit menos significativo

implica una estructura lógica

diseño automático de sistemas

• • • • • • • •

sumador genérico (ii)

```
library IEEE; use IEEE.std_logic_1164.all;

entity adder is
  generic( n : integer := 8 );
  port(
    x, y : in std_logic( n-1 downto 0 );
    cin : in std_logic;
    s : out std_logic_vector( n-1 downto 0 );
    cout : out std_logic );
end adder;

architecture syn of adder is
  function xor( constant x : in std_logic_vector ) return std_logic is
    variable y : std_logic := '0';
  begin
    for i in x'range loop
      y := y xor x(i);
    end loop;
    return y;
  end xor;
  signal c : std_logic_vector( n downto 0 );
begin
  c(0) <= cin;
  for i in s'reverse_range generate
    s(i) <= xor( x(i)&y(i)&c(i) );
    c(i+1) <= (x(i) and y(i)) or ( xor( x(i)&y(i) ) and c(i));
  end generate;
  cout <= c(n);
end syn;
```

esta función especifica el comportamiento
de una puerta xor genérica de n entradas

cada llamada es una instancia
diferente del HW combinacional
que implementa la función

diseño automático de sistemas

• • • • • • • •

sumador genérico (iii)

```

library IEEE; use IEEE.std_logic_1164.all;

entity adder is
  generic( n : integer := 8 );
  port(
    x, y : in std_logic( n-1 downto 0 );
    cin : in std_logic;
    s : out std_logic_vector( n-1 downto 0 );
    cout : out std_logic );
end adder;

architecture syn of adder is
begin
  process( x, y, cin )
    variable c : std_logic;
  begin
    c := cin;
    for i in s'reverse_range loop
      s(i) <= x(i) xor y(i) xor c;
      c := (x(i) and y(i)) or ((x(i) xor y(i)) and c);
    end loop;
    cout := c;
  end process;
end syn;

```

variable intermedia que propaga
el acarreo de una iteración a otra

comienza a calcular por el bit menos significativo

diseño automático de sistemas

• • • • • • • •

sumador genérico (iv)

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

entity adder is
  generic( n : integer := 8 );
  port(
    x, y : in std_logic_vector( n-1 downto 0 );
    cin : in std_logic;
    s : out std_logic_vector( n-1 downto 0 );
    cout : out std_logic );
end adder;

architecture syn of adder is
  signal temp : std_logic_vector( n downto 0 );
begin
  temp <= ( '0' & x ) + y + std_logic_vector(cin);
  s <= temp(n-1 downto 0);
  cout <= temp(n);
end syn;

```

Cuando se desee obtener el bit de acarreo
resultante de una operación de suma o resta
deberá extenderse 1 bit el operando más ancho.

esta especificación no implica
ninguna estructura lógica

Recuérdese que el resultado de la suma definida
en std_logic_unsigned tiene la misma anchura
que el mayor de los argumentos

diseño automático de sistemas

• • • • • • • •

sumador genérico (v)

```
library IEEE;
use IEEE.std_logic_1164.all;

package rtModules is
  procedure rippleAdder(
    constant x, y: in std_logic_vector; constant cin : in std_logic;
    signal s : out std_logic_vector; signal cout : out std_logic );
end rtModules;

package body rtModules is
  procedure rippleAdder(
    constant x, y: in std_logic_vector; constant cin : in std_logic;
    signal s : out std_logic_vector; signal cout : out std_logic )
  is
    variable c : std_logic;
  begin
    c := cin;
    for i in 0 to s'high-s'low loop
      s(i+s'low) <= x(i+x'low) xor y(i+y'low) xor c;
      c := (x(i+x'low) and y(i+y'low)) or ((x(i+x'low) xor y(i+y'low)) and c);
    end loop;
    cout <= c;
  end;
end rtModules;
```

© J.M. Mendias, 2004

diseño automático de sistemas

• • • • • • • •

sumador genérico (vi)

```
library IEEE;
use IEEE.std_logic_1164.all;
use work.rtModules.all;

entity adder is
  generic( n : integer := 8 );
  port(
    x, y : in std_logic( n-1 downto 0 );
    cin : in std_logic;
    s : out std_logic_vector( n-1 downto 0 );
    cout : out std_logic );
end adder;

architecture syn of adder is
begin
  process( x, y, cin )
  begin
    rippleAdder( x, y, cin, s, cout );
  end process;
end syn;

architecture syn2 of adder is
begin
  rippleAdder( x, y, cin, s, cout );
end syn;
```

© J.M. Mendias, 2004

diseño automático de sistemas

• • • • • • • •

registro genérico (i)

```
library IEEE; use IEEE.std_logic_1164.all;

entity register is
  generic( n : integer := 8 );
  port(
    clk, rst, ld : in std_logic;
    din : in std_logic_vector( n-1 downto 0 );
    dout : out std_logic_vector( n-1 downto 0 ) );
end register;

architecture syn of register is
  signal cs, ns : std_logic_vector( n-1 downto 0 );
begin
  state:
    process( clk, rst )
    begin
      if rst='1' then
        cs <= (others=>'0');
      elsif clk'event and clk='1' then
        cs <= ns;
      end if;
    end process;
  next_state:
    ns <= din when ld='1' else cs;
  moore_output:
    dout <= cs;
end syn;
```

registro genérico (ii)

```
library IEEE; use IEEE.std_logic_1164.all;

entity register is
  generic( n : integer := 8 );
  port(
    clk, rst, ld : in std_logic;
    din : in std_logic_vector( n-1 downto 0 );
    dout : out std_logic_vector( n-1 downto 0 ) );
end register;

architecture syn of register is
begin
  process( clk, rst )
  begin
    if rst='1' then
      dout <= (others=>'0');
    elsif clk'event and clk='1' then
      if ld='1' then
        dout <= din;
      end if;
    end if;
  end process;
end syn;
```

registro genérico (iii)

```

library IEEE; use IEEE.std_logic_1164.all;

entity register is
  generic( n : integer := 8 );
  port(
    clk, rst, ld : in std_logic;
    din : in std_logic_vector( n-1 downto 0 );
    dout : out std_logic_vector( n-1 downto 0 );
  end register;

  architecture syn of register is
  begin
    process
    begin
      wait until (clk'event and clk='1');
      if rst='1' then
        dout <= (others=>'0');
      elsif ld='1' then
        dout <= din;
      end if;
    end process;
  end syn;

```

no es necesario, pero permite
que el sintetizador localice el reloj

el reset es ahora síncrono

diseño automático de sistemas

registro de desplazamiento genérico (i)

```

entity shiftRegister is
  generic( n : integer := 8 );
  port(
    clk, rst, sht : in std_logic;
    din : in std_logic;
    dout : out std_logic_vector( n-1 downto 0 );
  end shiftRegister;

  architecture syn of shiftRegister is
    signal cs, ns : std_logic_vector( n-1 downto 0 );
  begin
    state: process( clk, rst )
    begin
      if rst='1' then
        cs <= (others=>'0');
      elsif clk'event and clk='1' then
        cs <= ns;
      end if;
    end process;
    next_state: process( cs, sht, din )
    begin
      ns <= cs;
      if sht='1' then
        for i in ns'high downto ns'low+1 loop
          ns(i) <= cs(i-1);
        end loop;
        ns(0) <= din;
      end if;
    end process;
    moore_output: dout <= cs;
  end syn;

```

diseño automático de sistemas

registro de desplazamiento genérico (ii)

```
entity shiftRegister is
  generic( n : integer := 8 );
  port(
    clk, rst, sht : in std_logic;
    din : in std_logic;
    dout : out std_logic_vector( n-1 downto 0 ) );
end shiftRegister;

architecture syn of shiftRegister is
  signal cs : std_logic_vector( n-1 downto 0 );
begin
  process( clk, rst )
  begin
    if rst='1' then
      cs <= (others=>'0');
    elsif clk'event and clk='1' then
      if sht='1' then
        for i in cs'high downto cs'low+1 loop
          cs(i) <= cs(i-1);
        end loop;
        cs(0) <= din;
      end if;
    end if;
  end process;
  dout <= cs;
end syn;
```

al ser señales podría hacerse también al revés sin sobrescribir los valores actuales

diseño automático de sistemas

registro de desplazamiento genérico (iii)

```
entity shiftRegister is
  generic( n : integer := 8 );
  port(
    clk, rst, sht : in std_logic;
    din : in std_logic;
    dout : buffer std_logic_vector( n-1 downto 0 ) );
end shiftRegister;

architecture syn of shiftRegister is
begin
  process( clk, rst )
  begin
    if rst='1' then
      dout <= (others=>'0');
    elsif clk'event and clk='1' then
      if sht='1' then
        for i in ns'high downto ns'low+1 loop
          dout(i) <= dout(i-1);
        end loop;
        dout(0) <= din;
      end if;
    end if;
  end process;
end syn;
```

siendo de tipo *buffer*, es un puerto de salida que internamente puede ser leído; esto evita que se declare la señal intermedia cs

diseño automático de sistemas

registro genérico con salida en alta impedancia (i)

```
entity bufferRegister is
  generic( n : integer := 8 );
  port(
    clk, rst, ld, en : in std_logic;
    din : in std_logic_vector( n-1 downto 0 );
    dout : out std_logic_vector( n-1 downto 0 ) );
end bufferRegister;

architecture syn of register is
  signal cs, ns : std_logic_vector( n-1 downto 0 );
begin
  state: process( clk, rst )
  begin
    if rst='1' then
      cs <= (others=>'0');
    elsif clk'event and clk='1' then
      cs <= ns;
    end if;
  end process;
  next_state: ns <= din when ld='1' else cs;
  mealy_output: dout <= cs when en='1' else (others=>'Z');
end syn;
```

registro genérico con salida en alta impedancia (ii)

```
entity bufferRegister is
  generic( n : integer := 8 );
  port(
    clk, rst, ld, en : in std_logic;
    din : in std_logic_vector( n-1 downto 0 );
    dout : out std_logic_vector( n-1 downto 0 ) );
end bufferRegister;

architecture syn of register is
  signal cs : std_logic_vector( n-1 downto 0 );
begin
  process( clk, rst, en )
  begin
    dout <= (others=>'Z');
    if en='1' then
      dout <= cs;
    end if;
    if rst='1' then
      cs <= (others=>'0');
    elsif clk'event and clk='1' then
      if ld='1' then
        cs <= din;
      end if;
    end if;
  end process;
end syn;
```

la señal cs se necesita para distinguir el estado que puede inicializarse y cargarse de modo independiente a la salida que puede desabilitarse

registro genérico con salida en alta impedancia (iii)

```
process( clk, rst, en )
begin
  if en='0' then
    dout <= (others=>'Z');
  elsif rst='1' then
    dout <= (others=>'0');
  elsif clk'event and clk='1' then
    ...
  end if;
end process;
```

en debe de estar en la lista de sensibilidad ya que eventos asíncronos de *en* afectan a la salida

error: sólo se inicializa si *en* es '1'

```
process( clk, rst, en )
begin
  if rst='1' then
    dout <= (others=>'0');
  elsif en='0' then
    dout <= (others=>'Z');
  elsif clk'event and clk='1' then
    ...
  end if;
end process;
```

error: sólo carga nuevos valores si *en* es '1', luego no se permitiría la desactivación y carga de nuevos datos en paralelo

```
process( clk, rst, en )
begin
  if rst='1' then
    dout <= (others=>'0');
  elsif clk'event and clk='1' then
    ...
  elsif en='0' then
    dout <= (others=>'Z');
  end if;
end process;
```

error: no existe HW real capaz de detectar la ausencia de flanco

```
process( clk, rst, en )
begin
  if rst='1' then
    dout <= (others=>'0');
  elsif clk'event and clk='1' then
    ...
  end if;
  if en='0' then
    dout <= (others=>'Z');
  end if;
end process;
```

error: la última asignación en un proceso sobrescribe las anteriores, luego si *en* vale '1' ni resetea ni carga

diseño automático de sistemas

contador ascendente/descendente genérico (i)

```
entity counter is
  generic( n : integer := 8 );
  port(
    clk, rst, ld, inc, dec : in std_logic;
    din : in std_logic_vector( n-1 downto 0 );
    tca, tcd : out std_logic;
    dout : out std_logic_vector( n-1 downto 0 );
  );
end counter;

architecture syn of register is
  signal cs, ns : std_logic_vector( n-1 downto 0 );
begin
  state: process( clk, rst )
  begin
    if rst='1' then cs <= (others=>'0');
    elsif clk'event and clk='1' then cs <= ns;
    end if;
  end process;
  next_state: process( cs, ld, inc, dec, din )
  begin
    if ld='1' then ns <= din;
    elsif inc='1' then ns <= cs + 1;
    elsif dec='1' then ns <= cs - 1;
    else ns <= cs;
    end if;
  end process;
  moore_output: dout <= cs;
  mealy_output: block
    tca <= inc and cs = (others=>'1');
    tcd <= dec and cs = (others=>'0');
  end block;
end syn;
```

no es necesario detectar el final de cuenta

se especifican las prioridades de las entradas de control

diseño automático de sistemas

contador ascendente/descendente genérico (ii)

```
entity counter is
  generic( n : integer := 8 );
  port(
    clk, rst, ld, inc, dec : in std_logic;
    din : in std_logic_vector( n-1 downto 0 );
    tca, tcd : out std_logic;
    dout : out std_logic_vector( n-1 downto 0 );
  );
end counter;

architecture syn of counter is
  signal cs : std_logic_vector( n-1 downto 0 );
begin
  process( clk, rst, cs, ld, inc, dec, din )
  begin
    tca <= inc and cs = (others=>'1');
    tcd <= dec and cs = (others=>'0');
    dout <= cs;
    if rst='1' then
      cs <= (others=>'0');
    elsif clk'event and clk='1' then
      if ld='1' then
        cs <= din;
      elsif inc='1' then
        cs <= cs + 1;
      elsif dec='1' then
        cs <= cs - 1;
      end if;
    end if;
  end process;
end syn;
```

contador módulo ascendente/descendente genérico

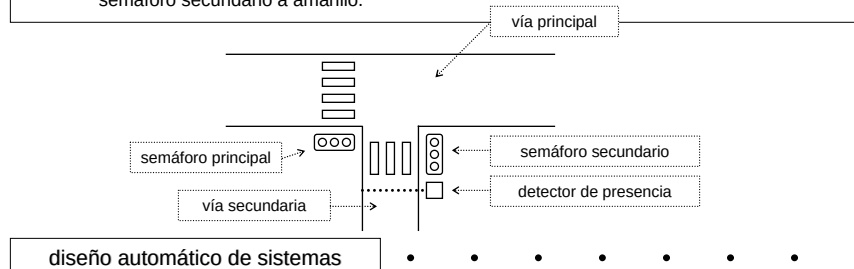
```
entity counter is
  generic(
    n : integer := 8;
    max_value, min_value : integer := 0 );
  port(
    clk, rst, ld, inc, dec : in std_logic;
    din : in std_logic_vector( n-1 downto 0 );
    tca, tcd : out std_logic;
    dout : out std_logic_vector( n-1 downto 0 );
  );
end counter;

architecture syn of counter is
  signal cs : std_logic_vector( n-1 downto 0 );
begin
  process( clk, rst, cs, ld, inc, dec, din )
  begin
    tca <= inc and cs = max_value;
    tcd <= dec and cs = min_value;
    dout <= cs;
    if rst='1' then
      cs <= conv_std_logic_vector( min_value, n );
    elsif clk'event and clk='1' then
      if ld='1' then
        cs <= din;
      elsif inc='1' then
        if cs = max_value then
          cs <= conv_std_logic_vector( min_value, n );
        else
          cs <= cs + 1;
        end if;
      elsif dec='1' then
        if cs = min_value then
          cs <= conv_std_logic_vector( max_value, n );
        else
          cs <= cs - 1;
        end if;
      end if;
    end if;
  end process;
end syn;
```

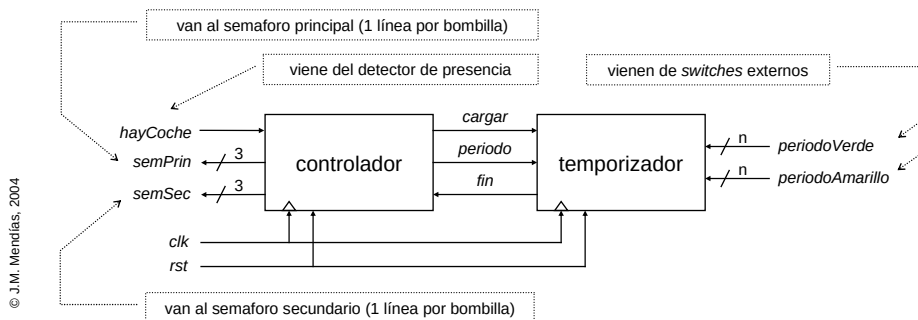
sistema algorítmico (i): especificación

- ☒ Se desea diseñar un sistema digital que controle 'inteligentemente' los semáforos que hay en el cruce entre una carretera principal y una secundaria, tal que la primera sea prioritaria sobre la segunda.
- Debe asegurarse que nunca ambos semáforos están en verde simultáneamente.
 - El semáforo principal estará verde como mínimo un *peridoVerde* y continuará en verde hasta que no se detecten coches en la vía secundaria.
 - Si hubiera coches en la vía secundaria, y sólo si el semáforo principal ha estado en verde durante un *peridoVerde* completo, el semáforo principal pasará a amarillo durante un *periodoAmarillo*, tras el cual se pondrá en rojo y el semáforo secundario en verde.
 - El semáforo secundario nunca permanecerá en verde más de un *peridoVerde*, tras el cual pasará a amarillo durante un *periodoAmarillo* y una vez finalizado éste, pasará a rojo. Entonces el semáforo principal pasará a verde.
 - No obstante si durante el *peridoVerde* del semáforo secundario la vía secundaria se quedara vacía, no se esperará a consumir el tiempo y se disparará la transición del semáforo secundario a amarillo.

© J.M. Mendias, 2004



sistema algorítmico (ii): estructura



© J.M. Mendias, 2004

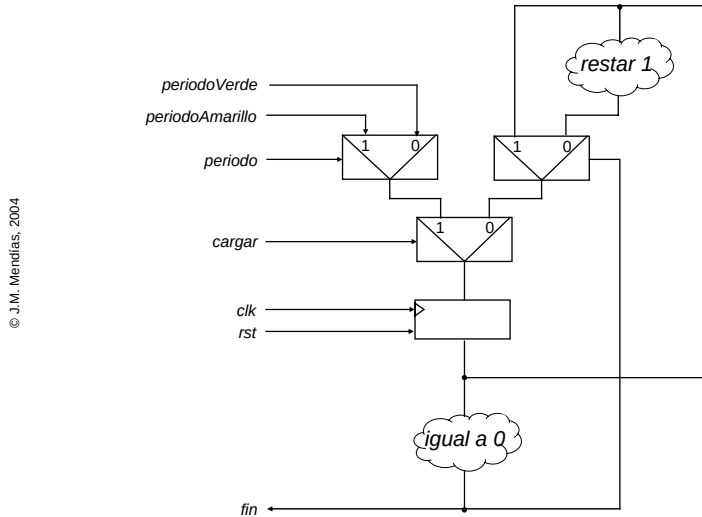
CONTROLADOR
Máquina de estados finitos
- reset asíncrono,
- 5 estados en correspondencia con el estado de los semáforos.

TEMPORIZADOR
Contador descendente no-cíclico con:
- reset asíncrono,
- carga síncrona seleccionable entre 2 valores
- con detector de fin de cuenta

diseño automático de sistemas

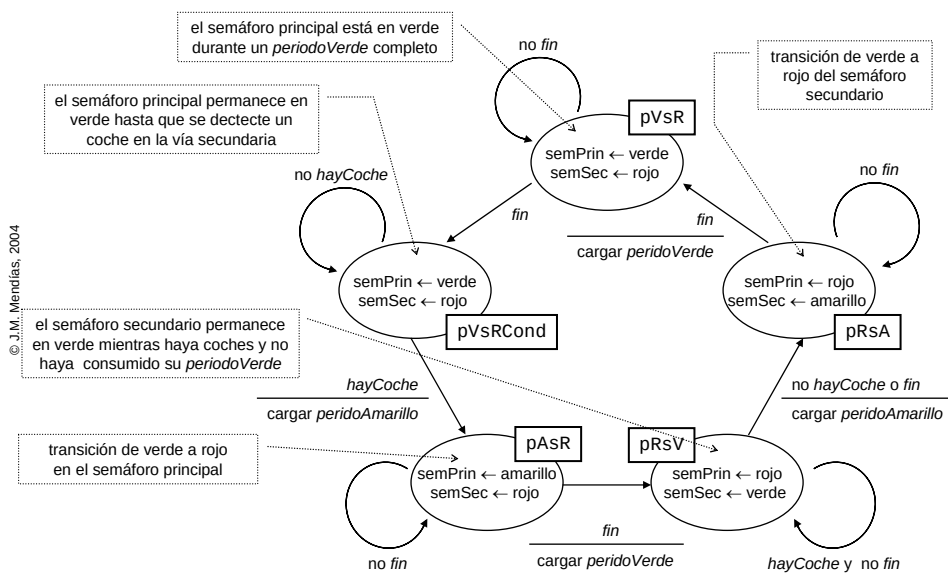
• • • • • • • •

sistema algorítmico (iii): temporizador RT



diseño automático de sistemas

sistema algorítmico (iv): controlador RT



diseño automático de sistemas

sistema algorítmico (v): interfaz VHDL

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

entity controlSemaforos is
  generic( n : integer := 16 );
  port(
    clk, rst : in std_logic;
    hayCoche : in std_logic;
    periodoVerde, periodoAmarillo : in std_logic_vector( n-1 downto 0 );
    semPrin, semSec : out std_logic_vector( 2 downto 0 );
  end controlSemaforos;

architecture syn of controlSemaforos is

  type estados_t is ( pVsR, pVsRCond, pAsR, pRsV, pRSA );
  constant rojo : std_logic_vector( 2 downto 0 ) := "100";
  constant amarillo : std_logic_vector( 2 downto 0 ) := "010";
  constant verde : std_logic_vector( 2 downto 0 ) := "001";

  signal cargar, periodo, fin : std_logic;
  signal csT, nsT : std_logic_vector( n-1 downto 0 );
  signal csC, nsC : estados_t;

begin
  ...
end controlSemaforos;

```

todos los datos numéricos son positivos, luego los objetos de tipo *std_logic_vector* deben interpretarse sin signo

declarando el estado del controlador como un tipo enumerado permite que la herramienta seleccione la codificación de estados más adecuada

el uso de constantes clarifica el código

estado (cuenta) del temporizador

estado del controlador

diseño automático de sistemas

sistema algorítmico (vi): temporizador VHDL

```

stateGenTemporizador:
process( csT, fin, cargar, periodo, periodoVerde, periodoAmarillo )
begin
  if cargar = '1' then
    if periodo = '1' then
      nsT <= periodoAmarillo;
    else
      nsT <= periodoVerde;
    end if;
  else
    if fin = '1' then
      nsT <= csT;
    else
      nsT <= csT - 1;
    end if;
  end if;
end process;

mooreGenTemporizador:
process( csT )
begin
  if csT = (others => '0') then
    fin <= '1';
  else
    fin <= '0';
  end if;
end process;

stateTemporizador:
process( clk, rst )
begin
  if rst = '1' then
    csT <= (others => '0');
  elsif clk'event and clk='1' then
    csT <= nsT;
  end if;
end process;

```

las asignaciones de *reset* asíncrono deben ser estáticas, si fuera síncrono podría inicializarse a *periodoVerde*

diseño automático de sistemas

sistema algorítmico (vii): controlador VHDL

```

stateGenControlador:
process( csC, hayCoche, fin )
begin
  case csC is
    when csC =>
      nsC <= csC;
      when pVSR =>
        if fin = '1' then
          nsC <= pVSRCond;
        end if;
      when pVSRCond =>
        if hayCoche = '1' then
          nsC <= pASR;
        end if;
      when pASR =>
        if fin = '1' then
          nsC <= pRSV;
        end if;
      when pRSV =>
        if hayCoche = '0' or fin = '1' then
          nsC <= pRSA;
        end if;
      when pRSA =>
        if fin = '1' then
          nsC <= pVsr;
        end if;
      end case;
  end process;

```

por defecto se permanece en el mismo estado

cuando se detectan cambios en las señales el estado cambia

sistema algorítmico (viii): controlador VHDL

```

mealyGenControlador:
process( csC, hayCoche, fin )
begin
  cargar <= '0';
  periodo <= '-';
  case csC is
    when pVSR =>
      null;
    when pVSRCond =>
      if hayCoche = '1' then
        cargar <= '1';
        periodo <= '1';
      end if;
    when pASR =>
      if fin = '1' then
        cargar <= '1';
        periodo <= '0';
      end if;
    when pRSV =>
      if hayCoche = '0' or fin = '1' then
        cargar <= '1';
        periodo <= '1';
      end if;
    when pRSA =>
      if fin = '1' then
        cargar <= '1';
        periodo <= '0';
      end if;
  end case;
end process;

```

por defecto el temporizador no debe cargar nada, ya que según su diseño la carga es prioritaria a la cuenta

si no carga el valor de periodo es indiferente

en el estado pVSR nunca se carga nada

sistema algorítmico (ix): controlador VHDL

```
mooreGenControlador:
process( csC )
begin
  case csC is
    when pVsR =>
      semPrin <= verde;
      semSec <= rojo;
    when pVsRCond =>
      semPrin <= verde;
      semSec <= rojo;
    when pAsR =>
      semPrin <= amarillo;
      semSec <= rojo;
    when pRsV =>
      semPrin <= rojo;
      semSec <= verde;
    when pRSA =>
      semPrin <= rojo;
      semSec <= amarillo;
  end case;
end process;
```

no hace falta asignar un valor por defecto para *semPrin* y *semSec* ya que se asignan en todos los casos

```
stateControlador:
process( clk, rst )
begin
  if rst = '1' then
    csC <= pVsR;
  elsif clk'event and clk='1' then
    csC <= nsC;
  end if;
end process;
```

sistema algorítmico (x): otro temporizador VHDL

```
temporizador:
process( clk, rst, csT, fin, cargar, periodo, periodoVerde, periodoAmarillo )
begin
  if csT = (others => '0') then
    fin <= '1';
  else
    fin <= '0';
  end if;
  if rst = '1' then
    csT <= (others => '0');
  elsif clk'event and clk='1' then
    if cargar = '1' then
      if periodo = '1' then
        csT <= periodoAmarillo;
      else
        csT <= periodoVerde;
      end if;
    else
      if fin = '1' then
        csT <= csT;
      else
        csT <= csT - 1;
      end if;
    end if;
  end if;
end process;
```

sistema algorítmico (xi): otro controlador VHDL

```

controlador:
process( clk, rst, csC, hayCoche, fin )
begin
  cargar <= '0'; periodo <= '-';
  case csC is
    when pVsR =>
      semPrin <= verde; semSec <= rojo;
    when pVsRCond =>
      semPrin <= verde; semSec <= rojo;
      if hayCoche = '1' then
        cargar <= '1'; periodo <= '1';
      end if;
    when pAsR =>
      semPrin <= amarillo; semSec <= rojo;
      if fin = '1' then
        cargar <= '1'; periodo <= '0';
      end if;
    when pRsV =>
      semPrin <= rojo; semSec <= verde;
      if hayCoche = '0' or fin = '1' then
        cargar <= '1'; periodo <= '1';
      end if;
    when pRsA =>
      semPrin <= rojo; semSec <= amarillo;
      if fin = '1' then
        cargar <= '1'; periodo <= '0';
      end if;
  end case;
  ...

```

```

...
if rst = '1' then
  csC <= pVsR;
elsif clk'event and clk='1' then
  case csC is
    when pVsR =>
      if fin = '1' then
        csC <= pVsRCond;
      end if;
    when pVsRCond =>
      if hayCoche = '1' then
        csC <= pAsR;
      end if;
    when pAsR =>
      if fin = '1' then
        csC <= pRsV;
      end if;
    when pRsV =>
      if hayCoche = '0' or fin = '1' then
        csC <= pRsA;
      end if;
    when pRsA =>
      if fin = '1' then
        csC <= pVsR;
      end if;
  end case;
end if;
end process;

```

diseño automático de sistemas

mezclando VHDL (i)

☒ No todo el diseño debe ser especificado en VHDL:

- las herramientas permiten mezclar código con otros mecanismos de especificación
 - ✓ esquemáticos, diagramas de estados, módulos prediseñados, uso de otros lenguajes
- desde el punto de vista VHDL:
 - ✓ estos comportamientos se encapsulan en bibliotecas
 - ✓ se instancian como componentes

☒ Ventajas:

- Se puede ahorrar tiempo y esfuerzo (discutible, excepto en caso de módulos prediseñados)

☒ Problemas:

- **pérdida de portabilidad:** VHDL "simulable" es estándar, VHDL "sintetizable" más o menos, los restantes mecanismos de especificación son dependientes de herramienta
- **dependencia tecnológica:** muchos módulos prediseñados pueden sólo ser aplicables para ciertas tecnologías objetivo.
- **necesidad de co-simulación:** se necesitan la interacción de varios simuladores cada uno especializado en una representación
 - ✓ existen generadores de modelos simulables VHDL

diseño automático de sistemas

mezclando VHDL (ii)

Componentes prediseñados

- ☒ **Tipos** de componentes prediseñados:
 - **Soft-macros:** especificaciones (esquemáticos o descripciones HDL) que se mezclan y sintetizan con el resto de los componentes del sistema
 - ✓ No se puede garantizar su rendimiento
 - **Hard-macros:** bloques presintetizados (típicamente netlist) que incluyen datos relativos a emplazamiento y rutado
 - ✓ Su rendimiento puede garantizarse
 - **Hardwired-macros:** bloques prefabricados y predifundidos sobre silicio
 - ✓ Su rendimiento está completamente caracterizado
- ☒ **Alternativas:** los componentes prediseñados pueden
 - estar almacenados en **bibliotecas de módulos** o crearse por un **generador de módulos**
 - tener pinout, funcionalidad y rendimiento **fijo** o **parametrizable**
 - la parametrización puede ser realizada por el **diseñador** o por la **herramientas** según las ligaduras de diseño
 - su uso puede ser **gratuito** o puede requerir el **pago de licencias** (según el uso)
 - pueden tener diversos grados de complejidad:
 - ✓ **Primitivos:** celdas elementales (AND, FF, ...) proyectables directamente sobre el HW
 - ✓ **IP-cores:** bloques de alta complejidad diseñados por compañías independientes
- ☒ Los componentes prediseñados pueden usarse y parametrizarse desde VHDL:
 - Instanciación directa
 - Inferencia a partir de un operador, una función o un fragmento de código que responda a una cierta estructura

diseño automático de sistemas

• • • • • • • •

mezclando VHDL (iii)

Directivas: atributos y pragmas

- ☒ La mayor parte de las herramientas pueden ser parcialmente controladas desde el propio código VHDL
 - Caracterización del entorno de funcionamiento del circuito
 - Ligaduras u opciones del proceso de síntesis
 - Control sobre el modo en que se interpretan las construcciones VHDL
- ☒ Habitualmente existe 2 maneras
 - mediante **atributos** VHDL


```
attribute OPT_MODE : string;
attribute OPT_MODE of cronometro : entity is "area"
```
 - mediante pragmas, comentarios VHDL que tienen un significado especial para la herramienta de síntesis


```
-- pragma translate_off
...
-- pragma translate_on
```

diseño automático de sistemas

• • • • • • • •